

Pharma Hands-on Exercises

Mike K Smith

Natalia Andriychuk
Rajni Bhaya

Samir Parmar
Taeyoon Kim

Leanne Vicente

2025-07-08

Table of contents

초록	4
1 데이터셋 읽기 및 탐색	5
1.1 목표	5
1.2 이 문서 사용법	5
1.3 데이터 소스	5
1.4 R 객체와 함수	5
1.5 미니 프로젝트 시작	5
2 연속형 및 범주형 데이터 요약	15
2.1 목표	15
2.2 이 문서 사용법	15
2.3 데이터 소스	15
2.4 R 객체와 함수	16
3 미니 프로젝트 시작	17
3.1 챌린지 1: 연령 그룹(AGEGRP) 추가	23
3.2 챌린지 2: 이상반응 데이터(adae.xpt) 활용	23
4 그룹별 요약 통계 산출	24
4.1 목표	24
4.2 이 문서 사용법	24
4.3 데이터 소스	24
5 미니 프로젝트 시작	25
5.1 챌린지 1: 체중(Weight) 및 신장(Height) 분석	28
6 요약표 데이터 통합	29
6.1 목표	29
6.2 데이터 소스	29
7 미니 프로젝트 시작	30
7.1 챌린지	33
8 {ggplot2}를 이용한 시각화 기초	34
8.1 목표	34
8.2 데이터 소스	34
8.3 1. 데이터셋 준비	34
8.4 2. ggplot 기본 구조	34
8.5 3. 기하학적 객체(Geom) 추가	35
8.6 4. 데이터 그룹화	36
8.7 5. 개별 피험자별 선 연결 (스파게티 플롯)	36
8.8 6. 레이블 및 테마 설정	37
8.9 7. 패싯(Facet)을 이용한 그룹별 분할	38
8.10 8. 결과 저장	38
8.11 9. 요약 통계(중앙값) 추가	38
8.12 10. 축 범위 조정 (Zooming)	39

8.13	챌린지	39
9	{ggplot2} 확장 패키지 활용	40
9.1	목표	40
9.2	데이터 소스	40
9.3	1. 데이터셋 준비	40
9.4	2. ALT 변화 추이 스파게티 플롯	41
9.5	3. {gghighlight}로 이상치 강조	41
9.6	4. 분포 시각화 (Boxplot & Jitter)	42
9.7	5. 분포 시각화 (Dotplot & Ridge plot)	43
9.8	6. {patchwork}를 이용한 그래프 배치	44
9.9	7. {cowplot}으로 워터마크 추가	45
9.10	챌린지	45
10	함수 작성 기초	46
10.1	1. 소개	46
10.2	2. 함수의 구조	47
10.3	3. 함수 생성 실습	47
10.4	4. 방어적 프로그래밍 (Defensive Programming)	48
10.5	5. 코드 리팩토링 (Refactoring)	48
10.6	6. 생략 부호 (Ellipses, ...) 활용	49
10.7	챌린지	49
11	반복 처리를 이용한 함수 적용	50
11.1	목표	50
11.2	1. 벡터화: 루프가 필수가 아닌 이유	50
11.3	2. 명시적 루프 (for loop) 사용하기	50
11.4	3. Apply 계열 함수	51
11.5	4. Tidy 반복 처리 (across & rowwise)	51
11.6	5. {purrr}와 map() 함수	52
11.7	6. 성능 최적화 (병렬 처리)	57
11.8	챌린지	57
12	재현 가능한 예제와 {reprex}	58
12.1	목표	58
12.2	1. Reprex란 무엇인가?	58
12.3	2. 기본 사용법	59
12.4	3. 재현 가능한 예제에 포함되어야 할 내용	59
12.5	4. 주의사항	59
12.6	챌린지	60
13	디버깅 (Debugging)	61
13.1	디버깅 전략	61
13.2	1. 깨끗한 환경 유지하기	61
13.3	2. R 다시 시작하기 (Restart R)	61
13.4	3. 실행 환경의 이해	61
13.5	4. debugonce() 활용하기	62
13.6	5. browser()와 중단점(Breakpoint)	62
13.7	6. 에러 추적: traceback()과 recover()	62
13.8	챌린지	62
14	Tidy Evaluation (타이디 평가)	64
14.1	1. tidyeval이란 무엇인가?	64
14.2	2. 평가된 인수와 인용된 인수	64
14.3	3. 열 충돌(Column Collision) 방지	64
14.4	4. 동적 점 (Dynamic Dots, ...)	65
14.5	5. Embrace 연산자 ({{ }})	65
14.6	6. Walrus 연산자 (:=)	66
14.7	7. 기호를 문자열로 변환 (as_name)	66

14.8 챌린지	66
15 단위 테스트 (Unit Testing)	67
15.1 목표	67
15.2 1. 테스트의 이점	67
15.3 2. {testthat}의 기대치(Expectation)	67
15.4 3. 테스트 가능한 코드 작성 예시	67
15.5 4. 테스트 코드 작성하기	68
15.6 5. 테스트 조직화 (Folder Structure)	68
15.7 챌린지	69
16 코드 스타일 (Code Style)	70
16.1 핵심 요약 (TL;DR)	70
16.2 1. 들여쓰기 정리 (단축키: Ctrl + I)	70
16.3 2. 코드 다시 포맷하기 (단축키: Ctrl + Shift + A)	70
16.4 3. 일관성 있는 스타일의 중요성	70
16.5 4. 주석 (Comments) 점검	71
16.6 5. 불필요한 라이브러리 로드 금지	71
16.7 6. 중복 배제 (DRY: Don't Repeat Yourself)	71
17 R 코드 스니펫 (Code Snippets)	72
17.1 1. 개요	72
17.2 2. 스니펫 사용법	72
17.3 3. 자주 쓰이는 내장 스니펫	72
17.4 4. 나만의 스니펫 만들기	72
17.5 5. 그래프 템플릿 스니펫	73
17.6 참고 자료	73

초록

이 책은 제약 데이터 분석을 위한 R의 실습 입문서를 제공합니다. 일련의 미니 프로젝트를 통해 데이터를 읽고, 탐색하고, 요약하고, 출판 품질의 표와 그림을 만들고, 자신만의 함수를 개발하는 방법을 배우게 됩니다. 이 책은 R 프로그래밍의 기초부터 tidy evaluation 및 자동화된 테스트와 같은 고급 개념에 이르기까지 광범위한 주제를 다룹니다. 이 책을 마치면 자신의 데이터 분석 프로젝트에 R을 사용할 수 있는 기술과 자신감을 갖게 될 것입니다.

각각 데이터 분석의 특정 측면에 초점을 맞춘 일련의 미니 프로젝트로 구성되어 있습니다. 프로젝트는 순서대로 완료되도록 설계되었으며, 각 프로젝트는 이전 프로젝트에서 배운 개념과 기술을 기반으로 합니다. 프로젝트는 다음을 포함하여 광범위한 주제를 다룹니다.

- **데이터 가져오기 및 탐색:** .xpt 파일을 포함한 다양한 파일 형식에서 데이터를 읽는 방법과 {tidyverse}의 함수를 사용하여 초기 데이터 탐색을 수행하는 방법을 배웁니다.
- **데이터 요약 및 표 작성:** 개수, 백분율 및 기술 통계를 포함하여 연속형 및 범주형 데이터의 요약 테이블을 만드는 방법을 배웁니다.
- **데이터 시각화:** {ggplot2} 패키지 및 해당 확장 기능을 사용하여 스파게티 플롯, 상자 그림 및 분포 플롯을 포함한 다양한 플롯을 만드는 방법을 배웁니다.
- **함수 작성 및 반복:** 반복적인 작업을 자동화하기 위해 자신만의 함수를 작성하는 방법과 루프 및 {purrr} 패키지를 사용하여 데이터에 함수를 반복적으로 적용하는 방법을 배웁니다.
- **재현 가능한 연구 및 디버깅:** {reprex} 패키지를 사용하여 재현 가능한 예제를 만드는 방법과 다양한 도구 및 기술을 사용하여 코드를 디버깅하는 방법을 배웁니다.
- **고급 주제:** tidy evaluation, {testthat}을 사용한 자동화된 테스트 및 코드 스타일과 같은 고급 주제에 대해 배웁니다.

Chapter 1

데이터셋 읽기 및 탐색

1.1 목표

- ADSL xpt 데이터셋 읽기
- 유효성 분석 대상군(Efficacy Population) 서브셋 추출
- 데이터 정렬
- 인구통계 리스팅 (Listing) 확인

1.2 이 문서 사용법

이 문서에는 코드 청크(밝은 회색 배경)와 설명 텍스트가 포함되어 있습니다. 이는 “Rmarkdown” 문서의 예시입니다. 문서 내에서 직접 코드를 작성하고 실행할 수 있으며, 결과는 각 코드 청크 바로 아래에 표시됩니다. 텍스트 지침에 따라 코드 청크를 수정하고 실행하여 원하는 출력을 생성해 보세요.

1.3 데이터 소스

이 프로젝트에서는 익명화된 CDISC 데이터셋을 사용합니다. 데이터는 다음 위치에서 찾을 수 있습니다:
<https://github.com/phuse-org/phuse-scripts/tree/master/data/adam/cdisc>

1.4 R 객체와 함수

R에서는 데이터, 벡터, 리스트 등 다양한 유형의 객체(Object)를 사용하며, 여기에 함수(Function)를 적용하여 작업을 수행합니다. 함수의 기본 구조는 `< _ >(< 1>= , < 2>= )` 입니다. 함수를 사용할 때 인수 이름을 생각하고 위치에 따라 인수를 전달할 수도 있습니다. 예를 들어 `myFunction(foo, 1, "bar")`에서 `foo`는 첫 번째 인수, `1`은 두 번째 인수, `"bar"`는 세 번째 인수로 전달됩니다. R 학습 초기에는 가급적 인수 이름을 명시적으로 사용하는 것을 권장합니다. RStudio IDE의 탭 완성 기능을 활용하면 편리하며, 함수의 자세한 인수를 확인하려면 콘솔에 `?< >`을 입력해 보세요.

1.5 미니 프로젝트 시작

1. 먼저 `tidyverse`, `rio`, `skimr`, `htmlTable` 패키지를 로드합니다. 패키지는 사용 전 로드가 필요하며, 보통 스크립트 시작 부분에서 로드하는 것이 좋습니다. 코드 청크 오른쪽의 초록색 화살표를 클릭하여 실행하세요.

```
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
Warning: package 'tibble' was built under R version 4.4.3
```

Warning: package 'purrr' was built under R version 4.4.2

Warning: package 'lubridate' was built under R version 4.4.2

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    3.5.2      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.0.4
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rio)
library(skimr)
library(htmlTable)
```

2. R에서 데이터를 읽어오는 방법은 다양합니다. `haven` 패키지의 `read_xpt()`, `readr` 패키지의 CSV/TSV 읽기 함수, `readxl` 패키지 등이 대표적입니다. 대부분의 함수는 첫 번째 인수로 데이터 경로를 사용합니다.

이 프로젝트에서는 `rio` 패키지를 사용합니다. `rio`는 파일 확장자를 통해 형식을 자동으로 판단하는 `import()` 함수를 제공하여 매우 편리합니다. GitHub의 ADSL 데이터를 읽어와 할당 연산자 `<-`를 사용해 객체에 저장해 보겠습니다.

```
adsl <- import(file = "./data/adsl.xpt")
```

3. `adsl` 객체의 상위 데이터를 확인하려면 객체 이름을 입력하거나 `head()` 함수를 사용합니다.

```
head(adsl)
```

	STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM					
1	CDISCPIL0T01	01-701-1015	1015	701	701	Placebo					
2	CDISCPIL0T01	01-701-1023	1023	701	701	Placebo					
3	CDISCPIL0T01	01-701-1028	1028	701	701	Xanomeline High Dose					
4	CDISCPIL0T01	01-701-1033	1033	701	701	Xanomeline Low Dose					
5	CDISCPIL0T01	01-701-1034	1034	701	701	Xanomeline High Dose					
6	CDISCPIL0T01	01-701-1047	1047	701	701	Placebo					
	TRT01P	TRT01PN		TRT01A	TRT01AN	TRTSDT					
1	Placebo	0		Placebo	0	2014-01-02					
2	Placebo	0		Placebo	0	2012-08-05					
3	Xanomeline High Dose	81	Xanomeline High Dose	81		2013-07-19					
4	Xanomeline Low Dose	54	Xanomeline Low Dose	54		2014-03-18					
5	Xanomeline High Dose	81	Xanomeline High Dose	81		2014-07-01					
6	Placebo	0		Placebo	0	2013-02-12					
	TRTEDT	TRTDUR	AVGDD	CUMDOSE	AGE	AGEGR1	AGEGR1N	AGEU	RACE	RACEN	SEX
1	2014-07-02	182	0.0	0	63	<65	1	YEARS	WHITE	1	F
2	2012-09-01	28	0.0	0	64	<65	1	YEARS	WHITE	1	M
3	2014-01-14	180	77.7	13986	71	65-80	2	YEARS	WHITE	1	M
4	2014-03-31	14	54.0	756	74	65-80	2	YEARS	WHITE	1	M
5	2014-12-30	183	76.9	14067	77	65-80	2	YEARS	WHITE	1	F
6	2013-03-09	26	0.0	0	85	>80	3	YEARS	WHITE	1	F
	ETHNIC	SAFFL	ITTFL	EFFFL	COMP8FL	COMP16FL	COMP24FL	DISCONFL			
1	HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y				
2	HISPANIC OR LATINO	Y	Y	Y	N	N	N				Y
3	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y				
4	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N				Y
5	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y				
6	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N				Y

	DSRAEFL	DTHFL	BMIBL	BMIBLGR1	HEIGHTBL	WEIGHTBL	EDUCLVL	DISONSDT	DURDIS
1			25.1	25-<30	147.3	54.4	16	2010-04-30	43.9
2	Y		30.4	>=30	162.6	80.3	14	2006-03-11	76.4
3			31.4	>=30	177.8	99.3	16	2009-12-16	42.8
4			28.8	25-<30	175.3	88.5	12	2009-08-02	55.3
5			26.1	25-<30	154.9	62.6	9	2011-09-29	32.9
6	Y		30.4	>=30	148.6	67.1	8	2009-07-26	42.0

	DURDSGR1	VISIT1DT	RFSTDTC	RFENDTC	VISNUMEN	RFENDT
1	>=12	2013-12-26	2014-01-02	2014-07-02	12	2014-07-02
2	>=12	2012-07-22	2012-08-05	2012-09-02	5	2012-09-02
3	>=12	2013-07-11	2013-07-19	2014-01-14	12	2014-01-14
4	>=12	2014-03-10	2014-03-18	2014-04-14	5	2014-04-14
5	>=12	2014-06-24	2014-07-01	2014-12-30	12	2014-12-30
6	>=12	2013-01-22	2013-02-12	2013-03-29	6	2013-03-29

	DCDECOD	DCREASCD	MMSETOT
1	COMPLETED	Completed	23
2	ADVERSE EVENT	Adverse Event	23
3	COMPLETED	Completed	23
4	STUDY TERMINATED BY SPONSOR	Sponsor Decision	23
5	COMPLETED	Completed	21
6	ADVERSE EVENT	Adverse Event	23

ads1 객체는 254행 48열로 구성되어 있습니다.

4. `sapply` 함수를 사용하여 각 열의 데이터 타입(character, numeric, factor 등)을 확인해 보겠습니다. `class` 함수는 객체의 타입을 알려줍니다.

```
sapply(X = ads1, FUN = class)
```

	STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM
"character"	TRT01P	TRT01PN	TRT01A	TRT01AN	TRTSDT	TRTEDT
"character"	TRTDUR	AVGDD	CUMDOSE	AGE	AGEGR1	AGEGR1N
"numeric"	AGEU	RACE	RACEN	SEX	ETHNIC	SAFFL
"character"	ITTFL	EFFFL	COMP8FL	COMP16FL	COMP24FL	DISCONFL
"character"	DSRAEFL	DTHFL	BMIBL	BMIBLGR1	HEIGHTBL	WEIGHTBL
"character"	EDUCLVL	DISONSDT	DURDIS	DURDSGR1	VISIT1DT	RFSTDTC
"numeric"	RFENDTC	VISNUMEN	RFENDT	DCDECOD	DCREASCD	MMSETOT
"character"						

5. 개별 변수의 타입을 확인하려면 `is.numeric()`, `is.character()` 등을 사용합니다. 특정 열을 참조할 때는 `< >$< >` 형식을 사용합니다.

중요: R은 대소문자를 구분(Case-sensitive)합니다! “AGE”와 “age”는 서로 다른 변수로 인식됩니다. SAS 데이터셋의 변수명은 주로 대문자임을 유의하세요.

```
is.numeric(ads1$AGE)
```

```
[1] TRUE
```

```
is.character(ads1$RACE)
```

```
[1] TRUE
```

6. `skimr` 패키지의 `skim()` 함수는 데이터의 요약 정보(행/열 개수, 결측치, 타입별 분포 등)를 빠르게 보여줍니다.


```
skim(data = adsl)
```

Table 1.1: Data summary

Name	adsl
Number of rows	254
Number of columns	48
Column type frequency:	
character	28
Date	5
numeric	15
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
STUDYID	0	1	12	12	0	1	0
USUBJID	0	1	11	11	0	254	0
SUBJID	0	1	4	4	0	254	0
SITEID	0	1	3	3	0	17	0
SITEGR1	0	1	3	3	0	11	0
ARM	0	1	7	20	0	3	0
TRT01P	0	1	7	20	0	3	0
TRT01A	0	1	7	20	0	3	0
AGEGR1	0	1	3	5	0	3	0
AGEU	0	1	5	5	0	1	0
RACE	0	1	5	32	0	3	0
SEX	0	1	1	1	0	2	0
ETHNIC	0	1	18	22	0	2	0
SAFFL	0	1	1	1	0	1	0
ITTFL	0	1	1	1	0	1	0
EFFFL	0	1	1	1	0	2	0
COMP8FL	0	1	1	1	0	2	0
COMP16FL	0	1	1	1	0	2	0
COMP24FL	0	1	1	1	0	2	0
DISCONFL	0	1	0	1	110	2	0
DSRAEFL	0	1	0	1	162	2	0
DTHFL	0	1	0	1	251	2	0
BMIBLGR1	0	1	3	6	0	3	0
DURDSGR1	0	1	3	4	0	2	0
RFSTDTC	0	1	10	10	0	205	0
RFENDTC	0	1	10	10	0	211	0
DCDECOD	0	1	5	27	0	9	0
DCREASCD	0	1	5	18	0	10	0

Variable type: Date

skim_variable	n_missing	complete_rate	min	max	median	n_unique
TRTSDT	0	1	2012-07-09	2014-09-02	2013-06-13	205
TRTEDT	0	1	2012-08-28	2015-03-05	2013-09-27	214
DISONSDT	0	1	1998-06-13	2013-09-16	2010-05-05	241
VISIT1DT	0	1	2012-07-06	2014-08-29	2013-06-02	207

skim_variable	n_missing	complete_rate	min	max	median	n_unique
RFENDT	0	1	2012-09-01	2015-03-05	2013-09-30	211

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
TRT01PN	0	1	44.65	33.84	0.0	0.00	54.00	81.00	81.0	
TRT01AN	0	1	44.65	33.84	0.0	0.00	54.00	81.00	81.0	
TRTDUR	0	1	116.09	70.30	1.0	46.25	133.50	183.00	212.0	
AVGDD	0	1	41.54	30.98	0.0	0.00	54.00	70.10	78.6	
CUMDOSE	0	1	4265.57	4963.57	0.0	0.00	2322.00	7809.75	15417.0	
AGE	0	1	75.09	8.25	51.0	70.00	77.00	81.00	89.0	
AGEGR1N	0	1	2.17	0.64	1.0	2.00	2.00	3.00	3.0	
RACEN	0	1	1.11	0.42	1.0	1.00	1.00	1.00	6.0	
BMIBL	1	1	24.67	4.09	13.7	21.90	24.20	27.30	40.1	
HEIGHTBL	0	1	163.93	10.76	135.9	156.20	162.85	171.50	195.6	
WEIGHTBL	1	1	66.65	14.13	34.0	55.30	66.70	77.10	108.0	
EDUCLVL	0	1	12.75	3.38	3.0	12.00	12.00	16.00	24.0	
DURDIS	0	1	43.94	28.40	2.2	24.30	36.25	57.35	183.1	
VISNUMEN	0	1	9.58	2.83	4.0	7.25	11.00	12.00	12.0	
MMSETOT	0	1	18.14	4.21	10.0	15.00	19.00	22.00	24.0	

참고: 출력 결과에 경고 메시지가 표시될 수 있습니다. 결측치나 데이터 타입 파싱 결과를 확인하는 용도로 활용하세요.

- 이제 유효성 분석 대상군(Efficacy Population) 추출을 위해 EFFFL 변수 값이 “Y”인 데이터만 필터링합니다. R에서 '같음'을 비교할 때는 == 연산자를 사용합니다.

```
adsl_eff <- filter(.data = adsl, EFFFL == 'Y')
head(adsl_eff)
```

	STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM					
1	CDISCPIL01	01-701-1015	1015	701	701	Placebo					
2	CDISCPIL01	01-701-1023	1023	701	701	Placebo					
3	CDISCPIL01	01-701-1028	1028	701	701	Xanomeline	High Dose				
4	CDISCPIL01	01-701-1033	1033	701	701	Xanomeline	Low Dose				
5	CDISCPIL01	01-701-1034	1034	701	701	Xanomeline	High Dose				
6	CDISCPIL01	01-701-1047	1047	701	701	Placebo					
	TRT01P	TRT01PN				TRT01A	TRT01AN	TRTSDT			
1	Placebo	0				Placebo	0	2014-01-02			
2	Placebo	0				Placebo	0	2012-08-05			
3	Xanomeline	High Dose	81	Xanomeline	High Dose	81	2013-07-19				
4	Xanomeline	Low Dose	54	Xanomeline	Low Dose	54	2014-03-18				
5	Xanomeline	High Dose	81	Xanomeline	High Dose	81	2014-07-01				
6	Placebo	0				Placebo	0	2013-02-12			
	TRTEDT	TRTDUR	AVGDD	CUMDOSE	AGE	AGEGR1	AGEGR1N	AGEU	RACE	RACEN	SEX
1	2014-07-02	182	0.0	0	63	<65	1	YEARS	WHITE	1	F
2	2012-09-01	28	0.0	0	64	<65	1	YEARS	WHITE	1	M
3	2014-01-14	180	77.7	13986	71	65-80	2	YEARS	WHITE	1	M
4	2014-03-31	14	54.0	756	74	65-80	2	YEARS	WHITE	1	M
5	2014-12-30	183	76.9	14067	77	65-80	2	YEARS	WHITE	1	F
6	2013-03-09	26	0.0	0	85	>80	3	YEARS	WHITE	1	F
	ETHNIC	SAFFL	ITTFL	EFFFL	COMP8FL	COMP16FL	COMP24FL	DISCONFL			
1	HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y	Y	Y		
2	HISPANIC OR LATINO	Y	Y	Y	N	N	N	N	Y		
3	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y	Y	Y		
4	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N	N	Y		

5	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y	
6	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N	Y
	DSRAEFL DTHFL BMIBL BMIBLGR1	HEIGHTBL	WEIGHTBL	EDUCLVL	DISONSDT	DURDIS		
1		25.1	25-<30	147.3	54.4	16	2010-04-30	43.9
2	Y	30.4	>=30	162.6	80.3	14	2006-03-11	76.4
3		31.4	>=30	177.8	99.3	16	2009-12-16	42.8
4		28.8	25-<30	175.3	88.5	12	2009-08-02	55.3
5		26.1	25-<30	154.9	62.6	9	2011-09-29	32.9
6	Y	30.4	>=30	148.6	67.1	8	2009-07-26	42.0
	DURDSGR1 VISIT1DT RFSTDTC	RFENDTC	VISNUMEN	RFENDT				
1	>=12	2013-12-26	2014-01-02	2014-07-02	12	2014-07-02		
2	>=12	2012-07-22	2012-08-05	2012-09-02	5	2012-09-02		
3	>=12	2013-07-11	2013-07-19	2014-01-14	12	2014-01-14		
4	>=12	2014-03-10	2014-03-18	2014-04-14	5	2014-04-14		
5	>=12	2014-06-24	2014-07-01	2014-12-30	12	2014-12-30		
6	>=12	2013-01-22	2013-02-12	2013-03-29	6	2013-03-29		
	DCDECOD	DCREASCD	MMSETOT					
1	COMPLETED	Completed	23					
2	ADVERSE EVENT	Adverse Event	23					
3	COMPLETED	Completed	23					
4	STUDY TERMINATED BY SPONSOR	Sponsor Decision	23					
5	COMPLETED	Completed	21					
6	ADVERSE EVENT	Adverse Event	23					

필터링 결과 234행의 데이터가 남았습니다. 대소문자와 따옴표 사용에 유의하세요.

8. 추출된 데이터를 USUBJID 기준으로 정렬합니다. arrange() 함수를 사용합니다.

```
sort_adsl_eff <- arrange(.data = adsl_eff, USUBJID)
head(sort_adsl_eff)
```

	STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM		
1	CDISCPILLOT01	01-701-1015	1015	701	701	Placebo		
2	CDISCPILLOT01	01-701-1023	1023	701	701	Placebo		
3	CDISCPILLOT01	01-701-1028	1028	701	701	Xanomeline High Dose		
4	CDISCPILLOT01	01-701-1033	1033	701	701	Xanomeline Low Dose		
5	CDISCPILLOT01	01-701-1034	1034	701	701	Xanomeline High Dose		
6	CDISCPILLOT01	01-701-1047	1047	701	701	Placebo		
	TRT01P	TRT01PN		TRT01A	TRT01AN	TRTSDT		
1	Placebo	0		Placebo	0	2014-01-02		
2	Placebo	0		Placebo	0	2012-08-05		
3	Xanomeline High Dose	81	Xanomeline High Dose	81	2013-07-19			
4	Xanomeline Low Dose	54	Xanomeline Low Dose	54	2014-03-18			
5	Xanomeline High Dose	81	Xanomeline High Dose	81	2014-07-01			
6	Placebo	0		Placebo	0	2013-02-12		
	TRTEDT	TRTDUR	AVGDD	CUMDOSE	AGE	AGEGR1	AGEGR1N	AGEU
1	2014-07-02	182	0.0	0	63	<65	1	YEARS
2	2012-09-01	28	0.0	0	64	<65	1	YEARS
3	2014-01-14	180	77.7	13986	71	65-80	2	YEARS
4	2014-03-31	14	54.0	756	74	65-80	2	YEARS
5	2014-12-30	183	76.9	14067	77	65-80	2	YEARS
6	2013-03-09	26	0.0	0	85	>80	3	YEARS
	ETHNIC	SAFFL	ITTFL	EFFFL	COMP8FL	COMP16FL	COMP24FL	DISCONFL
1	HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y	
2	HISPANIC OR LATINO	Y	Y	Y	N	N	N	Y
3	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y	
4	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N	Y
5	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y	
6	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N	Y
	DSRAEFL DTHFL BMIBL BMIBLGR1	HEIGHTBL	WEIGHTBL	EDUCLVL	DISONSDT	DURDIS		

1		25.1	25-<30	147.3	54.4	16	2010-04-30	43.9
2	Y	30.4	>=30	162.6	80.3	14	2006-03-11	76.4
3		31.4	>=30	177.8	99.3	16	2009-12-16	42.8
4		28.8	25-<30	175.3	88.5	12	2009-08-02	55.3
5		26.1	25-<30	154.9	62.6	9	2011-09-29	32.9
6	Y	30.4	>=30	148.6	67.1	8	2009-07-26	42.0
	DURDSGR1	VISIT1DT	RFSTDTC	RFENDTC	VISNUMEN	RFENDT		
1	>=12	2013-12-26	2014-01-02	2014-07-02	12	2014-07-02		
2	>=12	2012-07-22	2012-08-05	2012-09-02	5	2012-09-02		
3	>=12	2013-07-11	2013-07-19	2014-01-14	12	2014-01-14		
4	>=12	2014-03-10	2014-03-18	2014-04-14	5	2014-04-14		
5	>=12	2014-06-24	2014-07-01	2014-12-30	12	2014-12-30		
6	>=12	2013-01-22	2013-02-12	2013-03-29	6	2013-03-29		
	DCDECOD		DCREASCD	MMSETOT				
1	COMPLETED		Completed	23				
2	ADVERSE EVENT		Adverse Event	23				
3	COMPLETED		Completed	23				
4	STUDY TERMINATED BY SPONSOR		Sponsor Decision	23				
5	COMPLETED		Completed	21				
6	ADVERSE EVENT		Adverse Event	23				

tidyverse의 파이프 연산자 %>%를 사용하면 여러 단계를 체인처럼 연결할 수 있습니다.

```
sort_adsl_eff <- adsl %>%
  filter(EFFFL == "Y") %>%
  arrange(USUBJID)
```

위 코드는 “adsl 데이터를 가져와서(%>%) EFFFL이 Y인 것만 필터링하고(%>%) 다시 USUBJID 기준으로 정렬하라”고 읽을 수 있습니다.

9. 필터링된 adsl_eff 데이터의 요약을 다시 확인해 봅니다.

```
skim(adsl_eff)
```

Table 1.5: Data summary

Name	adsl_eff
Number of rows	234
Number of columns	48
Column type frequency:	
character	28
Date	5
numeric	15
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
STUDYID	0	1	12	12	0	1	0
USUBJID	0	1	11	11	0	234	0
SUBJID	0	1	4	4	0	234	0
SITEID	0	1	3	3	0	17	0
SITEGR1	0	1	3	3	0	11	0
ARM	0	1	7	20	0	3	0
TRT01P	0	1	7	20	0	3	0
TRT01A	0	1	7	20	0	3	0

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
AGEGR1	0	1	3	5	0	3	0
AGEU	0	1	5	5	0	1	0
RACE	0	1	5	32	0	3	0
SEX	0	1	1	1	0	2	0
ETHNIC	0	1	18	22	0	2	0
SAFFL	0	1	1	1	0	1	0
ITTFL	0	1	1	1	0	1	0
EFFFL	0	1	1	1	0	1	0
COMP8FL	0	1	1	1	0	2	0
COMP16FL	0	1	1	1	0	2	0
COMP24FL	0	1	1	1	0	2	0
DISCONFL	0	1	0	1	110	2	0
DSRAEFL	0	1	0	1	151	2	0
DTHFL	0	1	0	1	232	2	0
BMIBLGR1	0	1	3	6	0	3	0
DURDSGR1	0	1	3	4	0	2	0
RFSTDTC	0	1	10	10	0	190	0
RFENDTC	0	1	10	10	0	198	0
DCDECOD	0	1	5	27	0	9	0
DCREASCD	0	1	5	18	0	10	0

Variable type: Date

skim_variable	n_missing	complete_rate	min	max	median	n_unique
TRTSDT	0	1	2012-07-09	2014-09-02	2013-06-15	190
TRTEDT	0	1	2012-09-01	2015-03-05	2013-10-08	198
DISONSDT	0	1	1998-06-13	2013-09-16	2010-05-01	223
VISIT1DT	0	1	2012-07-06	2014-08-29	2013-06-05	191
RFENDT	0	1	2012-09-02	2015-03-05	2013-10-11	198

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
TRT01PN	0	1	44.31	33.55	0.0	0.00	54.00	81.00	81.0	
TRT01AN	0	1	44.31	33.55	0.0	0.00	54.00	81.00	81.0	
TRTDUR	0	1	123.89	67.54	2.0	58.25	163.50	183.00	212.0	
AVGDD	0	1	41.82	31.16	0.0	0.00	54.00	71.20	78.6	
CUMDOSE	0	1	4555.04	5056.10	0.0	0.00	2794.50	9531.00	15417.0	
AGE	0	1	75.01	8.13	51.0	70.25	76.50	81.00	88.0	
AGEGR1N	0	1	2.17	0.63	1.0	2.00	2.00	3.00	3.0	
RACEN	0	1	1.11	0.43	1.0	1.00	1.00	1.00	6.0	
BMIBL	1	1	24.79	4.06	13.7	21.90	24.60	27.50	40.1	
HEIGHTBL	0	1	164.27	10.77	135.9	156.20	163.65	172.70	195.6	
WEIGHTBL	1	1	67.19	13.89	34.0	55.80	67.10	77.10	108.0	
EDUCLVL	0	1	12.84	3.33	3.0	12.00	12.00	16.00	24.0	
DURDIS	0	1	44.74	28.82	2.2	24.55	36.60	58.38	183.1	
VISNUMEN	0	1	9.93	2.64	4.0	8.00	12.00	12.00	12.0	
MMSETOT	0	1	18.28	4.20	10.0	15.00	19.50	22.00	24.0	

10. 데이터를 TRT01A와 USUBJID 순으로 정렬하여 새로운 객체 `adsl_eff_srt`를 생성합니다.

```
adsl_eff_srt <- arrange(.data = adsl_eff, TRT01A, USUBJID)
head(adsl_eff_srt)
```

	STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM	TRT01P	TRT01PN		
1	CDISCPIL0T01	01-701-1015	1015	701	701	Placebo	Placebo	0		
2	CDISCPIL0T01	01-701-1023	1023	701	701	Placebo	Placebo	0		
3	CDISCPIL0T01	01-701-1047	1047	701	701	Placebo	Placebo	0		
4	CDISCPIL0T01	01-701-1118	1118	701	701	Placebo	Placebo	0		
5	CDISCPIL0T01	01-701-1130	1130	701	701	Placebo	Placebo	0		
6	CDISCPIL0T01	01-701-1153	1153	701	701	Placebo	Placebo	0		
	TRT01A	TRT01AN	TRTSDT	TRTEDT	TRTDUR	AVGDD	CUMDOSE	AGE	AGEGR1	AGEGR1N
1	Placebo	0	2014-01-02	2014-07-02	182	0	0	63	<65	1
2	Placebo	0	2012-08-05	2012-09-01	28	0	0	64	<65	1
3	Placebo	0	2013-02-12	2013-03-09	26	0	0	85	>80	3
4	Placebo	0	2014-03-12	2014-09-09	182	0	0	52	<65	1
5	Placebo	0	2014-02-15	2014-08-16	183	0	0	84	>80	3
6	Placebo	0	2013-09-23	2014-03-16	175	0	0	79	65-80	2
	AGEU	RACE	RACEN	SEX	ETHNIC	SAFFL	ITTFL	EFFFL	COMP8FL	
1	YEARS	WHITE	1	F	HISPANIC OR LATINO	Y	Y	Y	Y	
2	YEARS	WHITE	1	M	HISPANIC OR LATINO	Y	Y	Y	N	
3	YEARS	WHITE	1	F	NOT HISPANIC OR LATINO	Y	Y	Y	N	
4	YEARS	WHITE	1	M	NOT HISPANIC OR LATINO	Y	Y	Y	Y	
5	YEARS	WHITE	1	M	NOT HISPANIC OR LATINO	Y	Y	Y	Y	
6	YEARS	WHITE	1	F	NOT HISPANIC OR LATINO	Y	Y	Y	Y	
	COMP16FL	COMP24FL	DISCONFL	DSRAEFL	DTHFL	BMIBL	BMIBLGR1	HEIGHTBL	WEIGHTBL	
1	Y	Y				25.1	25-<30	147.3	54.4	
2	N	N	Y	Y		30.4	>=30	162.6	80.3	
3	N	N	Y	Y		30.4	>=30	148.6	67.1	
4	Y	Y				21.9	<25	180.3	71.2	
5	Y	Y				27.6	25-<30	169.7	79.4	
6	Y	Y				23.8	<25	156.2	58.1	
	EDUCLVL	DISONSDT	DURDIS	DURDSGR1	VISIT1DT	RFSTDTC	RFENDTC	VISNUMEN		
1	16	2010-04-30	43.9	>=12	2013-12-26	2014-01-02	2014-07-02	12		
2	14	2006-03-11	76.4	>=12	2012-07-22	2012-08-05	2012-09-02	5		
3	8	2009-07-26	42.0	>=12	2013-01-22	2013-02-12	2013-03-29	6		
4	14	2010-06-24	44.2	>=12	2014-02-27	2014-03-12	2014-09-09	12		
5	12	2010-05-22	44.7	>=12	2014-02-09	2014-02-15	2014-08-16	12		
6	6	2005-05-13	99.8	>=12	2013-09-06	2013-09-23	2014-04-01	12		
	RFENDT	DCDECOD	DCREASCD	MMSETOT						
1	2014-07-02	COMPLETED	Completed	23						
2	2012-09-02	ADVERSE EVENT	Adverse Event	23						
3	2013-03-29	ADVERSE EVENT	Adverse Event	23						
4	2014-09-09	COMPLETED	Completed	20						
5	2014-08-16	COMPLETED	Completed	19						
6	2014-04-01	COMPLETED	Completed	10						

11. select() 함수로 필요한 열(USUBJID, AGE, AGEU, SEX, RACE, ETHNIC, TRT01A)만 선택합니다.

```
adsl_eff_srt <- adsl_eff_srt %>%
  select(USUBJID, AGE, AGEU, SEX, RACE, ETHNIC, TRT01A)
head(adsl_eff_srt)
```

	USUBJID	AGE	AGEU	SEX	RACE	ETHNIC	TRT01A
1	01-701-1015	63	YEARS	F	WHITE	HISPANIC OR LATINO	Placebo
2	01-701-1023	64	YEARS	M	WHITE	HISPANIC OR LATINO	Placebo
3	01-701-1047	85	YEARS	F	WHITE NOT HISPANIC	OR LATINO	Placebo
4	01-701-1118	52	YEARS	M	WHITE NOT HISPANIC	OR LATINO	Placebo
5	01-701-1130	84	YEARS	M	WHITE NOT HISPANIC	OR LATINO	Placebo
6	01-701-1153	79	YEARS	F	WHITE NOT HISPANIC	OR LATINO	Placebo

이제 7개의 열만 남았습니다.

12. 출력용 리스팅을 위해 열 이름을 알아보기 쉽게 변경합니다. names() 함수에 c() 함수로 만든 이름

백터를 전달합니다.

```
names(x = adsl_eff_srt) <- c("Subject ID", "Age", "AgeUnits", "Sex", "Race",  
  ↪ "Ethnicity", "Treatment")  
head(adsl_eff_srt)
```

	Subject ID	Age	AgeUnits	Sex	Race	Ethnicity	Treatment
1	01-701-1015	63	YEARS	F	WHITE	HISPANIC OR LATINO	Placebo
2	01-701-1023	64	YEARS	M	WHITE	HISPANIC OR LATINO	Placebo
3	01-701-1047	85	YEARS	F	WHITE NOT	HISPANIC OR LATINO	Placebo
4	01-701-1118	52	YEARS	M	WHITE NOT	HISPANIC OR LATINO	Placebo
5	01-701-1130	84	YEARS	M	WHITE NOT	HISPANIC OR LATINO	Placebo
6	01-701-1153	79	YEARS	F	WHITE NOT	HISPANIC OR LATINO	Placebo

13. 최종 결과물을 출력합니다. Rmarkdown에서는 객체 이름을 입력하는 것만으로도 보기 좋은 HTML 표가 생성됩니다.

```
head(adsl_eff_srt)
```

	Subject ID	Age	AgeUnits	Sex	Race	Ethnicity	Treatment
1	01-701-1015	63	YEARS	F	WHITE	HISPANIC OR LATINO	Placebo
2	01-701-1023	64	YEARS	M	WHITE	HISPANIC OR LATINO	Placebo
3	01-701-1047	85	YEARS	F	WHITE NOT	HISPANIC OR LATINO	Placebo
4	01-701-1118	52	YEARS	M	WHITE NOT	HISPANIC OR LATINO	Placebo
5	01-701-1130	84	YEARS	M	WHITE NOT	HISPANIC OR LATINO	Placebo
6	01-701-1153	79	YEARS	F	WHITE NOT	HISPANIC OR LATINO	Placebo

외부 저장용으로 서식이 지정된 HTML 리스팅을 만들 때는 `htmlTable` 패키지 등을 활용할 수 있습니다. `align` 인수를 조정하여 정렬(l: 왼쪽, c: 중앙, r: 오른쪽)을 변경해 보세요.

```
htmlTable(x = adsl_eff_srt,  
  align = "l",  
  rnames = FALSE,  
  css.cell = "padding-left: .5em; padding-right: .2em;")
```

Chapter 2

연속형 및 범주형 데이터 요약

이 프로젝트에서는 다음과 같은 인구통계 요약표(Demography Summary Table) 생성을 목표로 합니다.

	AGEGR1	SEX	Placebo	Xanomeline Low Dose	Xanomeline High Dose
1	<65	Female	9 (64.3)	5 (62.5)	5 (45.5)
2	<65	Male	5 (35.7)	3 (37.5)	6 (54.5)
3	65-80	Female	22 (52.4)	28 (59.6)	28 (50.9)
4	65-80	Male	20 (47.6)	19 (40.4)	27 (49.1)
5	>80	Female	22 (73.3)	17 (58.6)	7 (38.9)
6	>80	Male	8 (26.7)	12 (41.4)	11 (61.1)

Figure 2.1: 인구통계 표

구조를 모방하여 R 코딩에 익숙해지고, 성별 및 치료군별 빈도와 백분율을 계산해 보겠습니다.

2.1 목표

- ADSL SAS (xpt) 데이터셋 읽기
- 유효성 분석 대상군(Efficacy Population) 서브셋 추출
- 그룹별 빈도(n) 산출 및 총 N 대비 백분율 계산
- 표 작성을 위한 데이터 전치(Transpose)

2.2 이 문서 사용법

이 문서에는 코드 청크와 설명 텍스트가 포함되어 있습니다. 지침에 따라 코드 청크를 수정하고 실행하며 출력을 확인해 보세요.

2.3 데이터 소스

이 프로젝트에서는 익명화된 CDISC 데이터셋을 사용합니다: <https://github.com/phuse-org/phuse-scripts/tree/master/data/adam/cdisc>

2.4 R 객체와 함수

R에서는 다양한 유형의 객체를 사용하며 함수를 적용합니다. 기본 구조는 `< _ >(< 1>= , < 2>= )`입니다. RStudio IDE의 탭 완성 기능을 활용하면 인수를 쉽게 입력할 수 있습니다.

Chapter 3

미니 프로젝트 시작

1. 먼저 필요한 패키지인 tidyverse와 rio를 로드합니다.

```
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
Warning: package 'tibble' was built under R version 4.4.3
```

```
Warning: package 'purrr' was built under R version 4.4.2
```

```
Warning: package 'lubridate' was built under R version 4.4.2
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
v dplyr      1.1.4      v readr      2.1.5
```

```
v forcats    1.0.0      v stringr    1.5.1
```

```
v ggplot2     3.5.2      v tibble     3.3.0
```

```
v lubridate  1.9.4      v tidyr      1.3.1
```

```
v purrr       1.0.4
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rio)
```

2. 데이터를 읽어와 adsl 객체에 저장합니다. rio 패키지의 import() 함수를 사용합니다.

```
adsl <- import(file = "./data/adsl.xpt")
```

```
head(adsl)
```

	STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM					
1	CDISCPIL0T01	01-701-1015	1015	701	701	Placebo					
2	CDISCPIL0T01	01-701-1023	1023	701	701	Placebo					
3	CDISCPIL0T01	01-701-1028	1028	701	701	Xanomeline High Dose					
4	CDISCPIL0T01	01-701-1033	1033	701	701	Xanomeline Low Dose					
5	CDISCPIL0T01	01-701-1034	1034	701	701	Xanomeline High Dose					
6	CDISCPIL0T01	01-701-1047	1047	701	701	Placebo					
	TRT01P	TRT01PN		TRT01A	TRT01AN	TRTSDT					
1	Placebo	0		Placebo	0	2014-01-02					
2	Placebo	0		Placebo	0	2012-08-05					
3	Xanomeline High Dose	81	Xanomeline High Dose	81		2013-07-19					
4	Xanomeline Low Dose	54	Xanomeline Low Dose	54		2014-03-18					
5	Xanomeline High Dose	81	Xanomeline High Dose	81		2014-07-01					
6	Placebo	0		Placebo	0	2013-02-12					
	TRTEDT	TRTDUR	AVGDD	CUMDOSE	AGE	AGEGR1	AGEGR1N	AGEU	RACE	RACEN	SEX
1	2014-07-02	182	0.0	0	63	<65	1	YEARS	WHITE	1	F

2	2012-09-01	28	0.0	0	64	<65	1 YEARS WHITE	1	M
3	2014-01-14	180	77.7	13986	71	65-80	2 YEARS WHITE	1	M
4	2014-03-31	14	54.0	756	74	65-80	2 YEARS WHITE	1	M
5	2014-12-30	183	76.9	14067	77	65-80	2 YEARS WHITE	1	F
6	2013-03-09	26	0.0	0	85	>80	3 YEARS WHITE	1	F
ETHNIC SAFFL ITTFL EFFFFL COMP8FL COMP16FL COMP24FL DISCONFL									
1	HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y	Y	
2	HISPANIC OR LATINO	Y	Y	Y	N	N	N		Y
3	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y		
4	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N		Y
5	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y		
6	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N		Y
DSRAEFL DTHFL BMIBL BMIBLGR1 HEIGHTBL WEIGHTBL EDUCLVL DISONSDT DURDIS									
1		25.1	25-<30	147.3	54.4	16	2010-04-30	43.9	
2	Y	30.4	>=30	162.6	80.3	14	2006-03-11	76.4	
3		31.4	>=30	177.8	99.3	16	2009-12-16	42.8	
4		28.8	25-<30	175.3	88.5	12	2009-08-02	55.3	
5		26.1	25-<30	154.9	62.6	9	2011-09-29	32.9	
6	Y	30.4	>=30	148.6	67.1	8	2009-07-26	42.0	
DURDSGR1 VISIT1DT RFSTDTC RFENDTC VISNUMEN RFENDT									
1	>=12	2013-12-26	2014-01-02	2014-07-02	12	2014-07-02			
2	>=12	2012-07-22	2012-08-05	2012-09-02	5	2012-09-02			
3	>=12	2013-07-11	2013-07-19	2014-01-14	12	2014-01-14			
4	>=12	2014-03-10	2014-03-18	2014-04-14	5	2014-04-14			
5	>=12	2014-06-24	2014-07-01	2014-12-30	12	2014-12-30			
6	>=12	2013-01-22	2013-02-12	2013-03-29	6	2013-03-29			
DCDECOD DCREASCD MMSETOT									
1	COMPLETED	Completed	23						
2	ADVERSE EVENT	Adverse Event	23						
3	COMPLETED	Completed	23						
4	STUDY TERMINATED BY SPONSOR	Sponsor Decision	23						
5	COMPLETED	Completed	21						
6	ADVERSE EVENT	Adverse Event	23						

유효성 분석 대상군 추출을 위해 EFFFFL 변수가 “Y”인 데이터만 필터링합니다.

간단 퀴즈: 다음 중 변수 EFFFFL의 값이 “Y”인 데이터를 필터링하는 올바른 옵션은 무엇입니까?

- `filter(.data = adsl, EFFFFL = Y)`
- `filter(.data = adsl, EFFFFL == "Y")`
- `filter(.data = adsl, "EFFFFL" == "Y")`
- `filter(.data = adsl, EFFFFL = "y")`

정답을 아래 코드 청크에 적용해 보세요.

```
adsl_eff <- adsl %>%
  filter(EFFFFL == "Y")
head(adsl_eff)
```

	STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM
1	CDISCPIL0T01	01-701-1015	1015	701	701	Placebo
2	CDISCPIL0T01	01-701-1023	1023	701	701	Placebo
3	CDISCPIL0T01	01-701-1028	1028	701	701	Xanomeline High Dose
4	CDISCPIL0T01	01-701-1033	1033	701	701	Xanomeline Low Dose
5	CDISCPIL0T01	01-701-1034	1034	701	701	Xanomeline High Dose
6	CDISCPIL0T01	01-701-1047	1047	701	701	Placebo
	TRT01P	TRT01PN		TRT01A	TRT01AN	TRTSDT
1	Placebo	0		Placebo	0	2014-01-02
2	Placebo	0		Placebo	0	2012-08-05
3	Xanomeline High Dose	81	Xanomeline High Dose	81	2013-07-19	
4	Xanomeline Low Dose	54	Xanomeline Low Dose	54	2014-03-18	

	5 Xanomeline High Dose	81 Xanomeline High Dose	81 2014-07-01								
6	Placebo	0	Placebo 0 2013-02-12								
	TRTEDT	TRTDUR	AVGDD	CUMDOSE	AGE	AGEGR1	AGEGR1N	AGEU	RACE	RACEN	SEX
1	2014-07-02	182	0.0	0	63	<65	1	YEARS	WHITE	1	F
2	2012-09-01	28	0.0	0	64	<65	1	YEARS	WHITE	1	M
3	2014-01-14	180	77.7	13986	71	65-80	2	YEARS	WHITE	1	M
4	2014-03-31	14	54.0	756	74	65-80	2	YEARS	WHITE	1	M
5	2014-12-30	183	76.9	14067	77	65-80	2	YEARS	WHITE	1	F
6	2013-03-09	26	0.0	0	85	>80	3	YEARS	WHITE	1	F
	ETHNIC	SAFFL	ITTFL	EFFFL	COMP8FL	COMP16FL	COMP24FL	DISCONFL			
1	HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y	Y			
2	HISPANIC OR LATINO	Y	Y	Y	N	N	N				Y
3	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y				
4	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N				Y
5	NOT HISPANIC OR LATINO	Y	Y	Y	Y	Y	Y				
6	NOT HISPANIC OR LATINO	Y	Y	Y	N	N	N				Y
	DSRAEFL	DTHFL	BMIBL	BMIBLGR1	HEIGHTBL	WEIGHTBL	EDUCLVL	DISONSDT	DURDIS		
1			25.1	25-<30	147.3	54.4	16	2010-04-30	43.9		
2	Y		30.4	>=30	162.6	80.3	14	2006-03-11	76.4		
3			31.4	>=30	177.8	99.3	16	2009-12-16	42.8		
4			28.8	25-<30	175.3	88.5	12	2009-08-02	55.3		
5			26.1	25-<30	154.9	62.6	9	2011-09-29	32.9		
6	Y		30.4	>=30	148.6	67.1	8	2009-07-26	42.0		
	DURDSGR1	VISIT1DT	RFSTDTC	RFENDTC	VISNUMEN	RFENDT					
1	>=12	2013-12-26	2014-01-02	2014-07-02	12	2014-07-02					
2	>=12	2012-07-22	2012-08-05	2012-09-02	5	2012-09-02					
3	>=12	2013-07-11	2013-07-19	2014-01-14	12	2014-01-14					
4	>=12	2014-03-10	2014-03-18	2014-04-14	5	2014-04-14					
5	>=12	2014-06-24	2014-07-01	2014-12-30	12	2014-12-30					
6	>=12	2013-01-22	2013-02-12	2013-03-29	6	2013-03-29					
	DCDECOD	DCREASCD	MMSETOT								
1	COMPLETED	Completed	23								
2	ADVERSE EVENT	Adverse Event	23								
3	COMPLETED	Completed	23								
4	STUDY TERMINATED BY SPONSOR	Sponsor Decision	23								
5	COMPLETED	Completed	21								
6	ADVERSE EVENT	Adverse Event	23								

팁: 대소문자가 확실하지 않을 때는 `casefold(EFFFL, upper=TRUE) == "Y"`와 같이 처리하는 것이 안전합니다.

3. 주변 합계(Marginal Totals, N) 산출 - 각 치료군별 대상자 수 계산.

백분율 계산의 분모가 될 큰 N을 산출합니다. `group_by()`와 `count()`를 사용합니다.

SAS 사용자를 위한 팁: `group_by()`는 SAS의 BY 문과 유사하게 작동합니다. 이후의 모든 작업은 그룹별로 수행됩니다.

```
adsl_eff %>%
  group_by(TRT01A) %>%
  count(name = "N")
```

```
# A tibble: 3 x 2
# Groups:   TRT01A [3]
  TRT01A      N
  <chr>    <int>
1 Placebo      79
2 Xanomeline High Dose 74
3 Xanomeline Low Dose 81
```

정렬을 위해 `TRT01AN(숫자)`과 `TRT01A(레이블)`를 함께 그룹화하는 것이 유용합니다.

```
Big_N_cnt <- adsl_eff %>%
  group_by(TRT01AN, TRT01A) %>%
  count(name = "N")
head(Big_N_cnt)
```

```
# A tibble: 3 x 3
# Groups:   TRT01AN, TRT01A [3]
  TRT01AN TRT01A      N
    <dbl> <chr>    <int>
1      0 Placebo     79
2     54 Xanomeline Low Dose 81
3     81 Xanomeline High Dose 74
```

4. 치료군 내 성별(SEX) 빈도(small n) 산출.

그룹화 변수에 SEX를 추가합니다.

```
small_n_cnt <- adsl_eff %>%
  group_by(TRT01AN, TRT01A, SEX) %>%
  count(name = "n")
head(small_n_cnt)
```

```
# A tibble: 6 x 4
# Groups:   TRT01AN, TRT01A, SEX [6]
  TRT01AN TRT01A      SEX      n
    <dbl> <chr>    <chr> <int>
1      0 Placebo      F      46
2      0 Placebo      M      33
3     54 Xanomeline Low Dose F      47
4     54 Xanomeline Low Dose M      34
5     81 Xanomeline High Dose F      35
6     81 Xanomeline High Dose M      39
```

5. 백분율 계산을 위한 데이터 병합.

left_join()을 사용하여 빈도 데이터(small_n_cnt)와 합계 데이터(Big_N_cnt)를 결합합니다. SQL의 JOIN과 유사합니다.

```
adsl_mrg_cnt <- small_n_cnt %>%
  left_join(Big_N_cnt, by = c("TRT01A", "TRT01AN"))
head(adsl_mrg_cnt)
```

```
# A tibble: 6 x 5
# Groups:   TRT01AN, TRT01A, SEX [6]
  TRT01AN TRT01A      SEX      n      N
    <dbl> <chr>    <chr> <int> <int>
1      0 Placebo      F      46     79
2      0 Placebo      M      33     79
3     54 Xanomeline Low Dose F      47     81
4     54 Xanomeline Low Dose M      34     81
5     81 Xanomeline High Dose F      35     74
6     81 Xanomeline High Dose M      39     74
```

6. 백분율 산출.

mutate() 함수로 새로운 열 perc를 만듭니다.

```
adsl_mrg_cnt %>%
  mutate(perc = (n/N)*100)
```

```
# A tibble: 6 x 6
# Groups:   TRT01AN, TRT01A, SEX [6]
  TRT01AN TRT01A      SEX      n      N perc
    <dbl> <chr>    <chr>      n      N  <dbl>
```

	<dbl>	<chr>		<chr>	<int>	<int>	<dbl>
1	0	Placebo	F		46	79	58.2
2	0	Placebo	M		33	79	41.8
3	54	Xanomeline Low Dose	F		47	81	58.0
4	54	Xanomeline Low Dose	M		34	81	42.0
5	81	Xanomeline High Dose	F		35	74	47.3
6	81	Xanomeline High Dose	M		39	74	52.7

7. 소수점 반올림.

round() 함수를 사용합니다.

주의: R의 round()는 'Round to Even' 방식을 따르므로 SAS와 결과가 다를 수 있습니다. 정밀한 비교가 필요한 경우 주의하세요.

```
adsl_mrg_cnt %>%
  mutate(perc = round((n/N)*100, digits=1))
```

```
# A tibble: 6 x 6
# Groups:   TRT01AN, TRT01A, SEX [6]
  TRT01AN TRT01A      SEX      n      N perc
  <dbl> <chr>      <chr> <int> <int> <dbl>
1      0 Placebo      F      46     79 58.2
2      0 Placebo      M      33     79 41.8
3     54 Xanomeline Low Dose F      47     81 58
4     54 Xanomeline Low Dose M      34     81 42
5     81 Xanomeline High Dose F      35     74 47.3
6     81 Xanomeline High Dose M      39     74 52.7
```

8. 수치 데이터 서식 지정.

표에 표시할 때 "20.0"과 같이 일관된 소수점 자릿수를 보여주기 위해 format(nsmall=1)을 사용합니다. 수치 데이터가 문자(character) 타입으로 변환됩니다.

```
adsl_mrg_cnt %>%
  mutate(perc = round((n/N)*100, digits=1)) %>%
  mutate(perc_char = format(perc, nsmall=1))
```

```
# A tibble: 6 x 7
# Groups:   TRT01AN, TRT01A, SEX [6]
  TRT01AN TRT01A      SEX      n      N perc perc_char
  <dbl> <chr>      <chr> <int> <int> <dbl> <chr>
1      0 Placebo      F      46     79 58.2 58.2
2      0 Placebo      M      33     79 41.8 41.8
3     54 Xanomeline Low Dose F      47     81 58 58.0
4     54 Xanomeline Low Dose M      34     81 42 42.0
5     81 Xanomeline High Dose F      35     74 47.3 47.3
6     81 Xanomeline High Dose M      39     74 52.7 52.7
```

9. 빈도와 백분율 합치기.

paste() 또는 paste0()를 사용하여 "n (perc%)" 형식을 만듭니다.

```
adsl_mrg_cnt %>%
  mutate(perc = round((n/N)*100, digits=1)) %>%
  mutate(perc_char = format(perc, nsmall=1)) %>%
  mutate(npct = paste(n, paste0("(", perc_char, ")")))
```

```
# A tibble: 6 x 8
# Groups:   TRT01AN, TRT01A, SEX [6]
  TRT01AN TRT01A      SEX      n      N perc perc_char npct
  <dbl> <chr>      <chr> <int> <int> <dbl> <chr>      <chr>
1      0 Placebo      F      46     79 58.2 58.2      46 (58.2)
```

2	0 Placebo	M	33	79	41.8	41.8	33 (41.8)
3	54 Xanomeline Low Dose	F	47	81	58	58.0	47 (58.0)
4	54 Xanomeline Low Dose	M	34	81	42	42.0	34 (42.0)
5	81 Xanomeline High Dose	F	35	74	47.3	47.3	35 (47.3)
6	81 Xanomeline High Dose	M	39	74	52.7	52.7	39 (52.7)

10. 성별 레이블 재코딩.

“M”, “F”를 “Male”, “Female”로 변경합니다. `recode()` 함수를 활용합니다.

```
adsl_mrg_cnt %>%
  mutate(perc = round((n/N)*100, digits=1)) %>%
  mutate(perc_char = format(perc, nsmall=1)) %>%
  mutate(npct = paste(n, paste0("(", perc_char, ")"))) %>%
  mutate(SEX = recode(SEX, "M" = "Male", "F" = "Female"))
```

A tibble: 6 x 8

Groups: TRT01AN, TRT01A, SEX [6]

	TRT01AN	TRT01A	SEX	n	N	perc	perc_char	npct
	<dbl>	<chr>	<chr>	<int>	<int>	<dbl>	<chr>	<chr>
1	0	Placebo	Female	46	79	58.2	58.2	46 (58.2)
2	0	Placebo	Male	33	79	41.8	41.8	33 (41.8)
3	54	Xanomeline Low Dose	Female	47	81	58	58.0	47 (58.0)
4	54	Xanomeline Low Dose	Male	34	81	42	42.0	34 (42.0)
5	81	Xanomeline High Dose	Female	35	74	47.3	47.3	35 (47.3)
6	81	Xanomeline High Dose	Male	39	74	52.7	52.7	39 (52.7)

11. 그룹 해제.

이후 작업에 영향을 주지 않도록 `ungroup()`을 수행합니다.

```
adsl_mrg_cnt %>%
  ungroup()
```

A tibble: 6 x 5

	TRT01AN	TRT01A	SEX	n	N
	<dbl>	<chr>	<chr>	<int>	<int>
1	0	Placebo	F	46	79
2	0	Placebo	M	33	79
3	54	Xanomeline Low Dose	F	47	81
4	54	Xanomeline Low Dose	M	34	81
5	81	Xanomeline High Dose	F	35	74
6	81	Xanomeline High Dose	M	39	74

12. 파이프를 이용한 워크플로 통합.

위의 모든 과정을 하나의 파이프라인으로 연결합니다.

```
adsl_mrg_cnt <- small_n_cnt %>%
  left_join(Big_N_cnt, by = c("TRT01A", "TRT01AN")) %>%
  mutate(perc = round(n/N*100, digits=1)) %>%
  mutate(perc_char = format(perc, nsmall=1)) %>%
  mutate(npct = paste(n, paste0("(", perc_char, ")"))) %>%
  mutate(SEX = recode(SEX, "M" = "Male", "F" = "Female")) %>%
  ungroup() %>%
  select(TRT01A, SEX, npct)

head(adsl_mrg_cnt)
```

A tibble: 6 x 3

	TRT01A	SEX	npct
	<chr>	<chr>	<chr>
1	Placebo	Female	46 (58.2)

```

2 Placebo           Male    33 (41.8)
3 Xanomeline Low Dose Female 47 (58.0)
4 Xanomeline Low Dose Male   34 (42.0)
5 Xanomeline High Dose Female 35 (47.3)
6 Xanomeline High Dose Male   39 (52.7)

```

13. 데이터 전치 (Wide Format).

표 형식에 맞게 `pivot_wider()`를 사용하여 데이터를 펼칩니다.

```

adsl_mrg_cnt %>%
  pivot_wider(names_from = TRT01A, values_from = npct)

# A tibble: 2 x 4
  SEX    Placebo `Xanomeline Low Dose` `Xanomeline High Dose`
  <chr>  <chr>      <chr>                        <chr>
1 Female 46 (58.2) 47 (58.0)                    35 (47.3)
2 Male   33 (41.8) 34 (42.0)                    39 (52.7)

```

3.1 챌린지 1: 연령 그룹(AGEGRP) 추가

위 단계를 반복하되, 그룹화 변수에 AGEGRP1을 추가하여 분석해 보세요.

3.2 챌린지 2: 이상반응 데이터(adae.xpt) 활용

adae.xpt 데이터를 읽어와 안전성 분석 대상군(SAFFL)에 대한 신체기관별 이상반응(AEBODSYS) 빈도와 백분율을 산출해 보세요.

Chapter 4

그룹별 요약 통계 산출

이 프로젝트에서는 연속형 변수에 대한 그룹별 요약 통계(Summary Statistics)를 생성하고, 이를 인구통계 요약표 형식에 맞춰 전치하는 방법을 배웁니다.

4.1 목표

- ADSL SAS (xpt) 데이터셋 읽기
- 유효성 분석 대상군(Efficacy Population) 서브셋 추출
- 그룹화 변수별 요약 통계 산출
- 표 작성을 위한 데이터 전치(Transpose)

4.2 이 문서 사용법

지침에 따라 코드 청크를 수정하고 실행하며 출력을 확인해 보세요.

4.3 데이터 소스

익명화된 CDISC 데이터셋: <https://github.com/phuse-org/phuse-scripts/tree/master/data/adam/cdisc>

Chapter 5

미니 프로젝트 시작

1. 먼저 필요한 패키지인 tidyverse와 rio를 로드합니다.

```
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
Warning: package 'tibble' was built under R version 4.4.3
```

```
Warning: package 'purrr' was built under R version 4.4.2
```

```
Warning: package 'lubridate' was built under R version 4.4.2
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
v dplyr      1.1.4      v readr      2.1.5
```

```
v forcats    1.0.0      v stringr    1.5.1
```

```
v ggplot2    3.5.2      v tibble     3.3.0
```

```
v lubridate  1.9.4      v tidyr      1.3.1
```

```
v purrr      1.0.4
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(rio)
```

2. 데이터를 읽어오고 유효성 분석 대상군을 필터링합니다. 성별 레이블도 함께 재코딩합니다.

```
adsl <- import(file =  
  ↪ "https://github.com/phuse-org/phuse-scripts/raw/master/data/adam/cdisc/adsl.xpt")
```

```
adsl_eff <- adsl %>%  
  filter( EFFFL == "Y" ) %>%  
  mutate(SEX = recode(SEX, "M" = "Male", "F" = "Female"))
```

3. 치료군 및 성별별 연령(AGE) 평균 산출.

summarize() 함수를 사용하여 요약 통계를 계산합니다. 이는 SAS의 PROC MEANS와 유사합니다.

질문: 3개의 치료군과 2개의 성별이 있을 때, TRT01AN, TRT01A, SEX로 그룹화하면 결과 데이터는 몇 행이 될까요?

```
adsl_eff %>%  
  group_by( TRT01AN, TRT01A, SEX ) %>%  
  summarize(mean = mean(AGE), .groups = 'drop')
```

```
# A tibble: 6 x 4
```

```
  TRT01AN TRT01A      SEX    mean
```

	<dbl>	<chr>		<chr>	<dbl>
1	0	Placebo	Female	76.1	
2	0	Placebo	Male	73.4	
3	54	Xanomeline Low Dose	Female	76.4	
4	54	Xanomeline Low Dose	Male	75.6	
5	81	Xanomeline High Dose	Female	74.3	
6	81	Xanomeline High Dose	Male	73.6	

평균값의 소수점 자릿수를 정리하고 문자형으로 변환해 보겠습니다.

```
adsl_eff %>%
  group_by( TRT01A, TRT01AN, SEX ) %>%
  summarize(mean = mean(AGE) %>%
    round(digits = 1) %>%
    format(nsmall = 1),
    .groups = 'drop')
```

```
# A tibble: 6 x 4
  TRT01A          TRT01AN SEX    mean
  <chr>          <dbl> <chr> <chr>
1 Placebo              0 Female 76.1
2 Placebo              0 Male   73.4
3 Xanomeline High Dose  81 Female 74.3
4 Xanomeline High Dose  81 Male   73.6
5 Xanomeline Low Dose   54 Female 76.4
6 Xanomeline Low Dose   54 Male   75.6
```

4. 다양한 요약 통계 산출.

표준편차(sd), 중앙값(median), 최솟값(min), 최대값(max), 빈도(n)를 함께 계산합니다.

```
age_stat <- adsl_eff %>%
  group_by(TRT01AN, TRT01A, SEX) %>%
  summarize(mean = mean(AGE) %>% round(digits = 1) %>% format(nsmall = 1),
    sd = sd(AGE) %>% round(digits = 1) %>% format(nsmall = 1),
    med = median(AGE) %>% format(nsmall = 0),
    min = min(AGE) %>% format(nsmall = 0),
    max = max(AGE) %>% format(nsmall = 0),
    n = n(),
    .groups = 'drop')

head(age_stat)
```

```
# A tibble: 6 x 9
  TRT01AN TRT01A          SEX    mean sd    med min  max    n
  <dbl> <chr>          <chr> <chr> <chr> <chr> <chr> <chr> <int>
1      0 Placebo      Female 76.1  8.5  77.5  59   88    46
2      0 Placebo      Male   73.4  8.1   74   52   85    33
3     54 Xanomeline Low Dose Female 76.4  7.6   78   56   87    47
4     54 Xanomeline Low Dose Male   75.6  8.7  77.5  51   88    34
5     81 Xanomeline High Dose Female 74.3  7.5   76   56   88    35
6     81 Xanomeline High Dose Male   73.6  8.3   75   56   86    39
```

5. 범위(Range) 형식 만들기.

min과 max를 결합하여 “(min, max)” 형태의 문자열을 만듭니다.

```
age_stat2 <- age_stat %>%
  mutate(range_minmax = paste0("(", min, ", ", max, ")"))

head(age_stat2)
```

```
# A tibble: 6 x 10
```

	TRT01A	TRT01A	SEX	mean	sd	med	min	max	n	range_minmax
	<dbl>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<int>	<chr>
1	0	Placebo	Fema~	76.1	8.5	77.5	59	88	46	(59, 88)
2	0	Placebo	Male	73.4	8.1	74	52	85	33	(52, 85)
3	54	Xanomeline Low~	Fema~	76.4	7.6	78	56	87	47	(56, 87)
4	54	Xanomeline Low~	Male	75.6	8.7	77.5	51	88	34	(51, 88)
5	81	Xanomeline Hig~	Fema~	74.3	7.5	76	56	88	35	(56, 88)
6	81	Xanomeline Hig~	Male	73.6	8.3	75	56	86	39	(56, 86)

6. 데이터 전치 - Long Format으로 변환.

각 통계량이 열로 나열된 현재 구조를, 통계량 이름(category)과 값(values)이 행으로 나열되는 긴 형식(Long Format)으로 바꿉니다. pivot_longer()를 사용합니다.

SAS 사용자를 위한 팁: select()는 SAS의 KEEP 문과 유사합니다.

```
desc_stat_long <- age_stat2 %>%
  select(TRT01A, SEX, n, mean, med, sd, range_minmax) %>%
  mutate(across(where(is.numeric), as.character)) %>%
  pivot_longer(~c(TRT01A, SEX), names_to = "category", values_to = "values")

head(desc_stat_long)
```

```
# A tibble: 6 x 4
  TRT01A SEX    category    values
  <chr>  <chr>  <chr>      <chr>
1 Placebo Female n          46
2 Placebo Female mean       76.1
3 Placebo Female med        77.5
4 Placebo Female sd          8.5
5 Placebo Female range_minmax (59, 88)
6 Placebo Male    n          33
```

7. 데이터 전치 - Wide Format으로 변환.

표 형식에 맞춰 치료군과 성별을 열로 배치합니다. pivot_wider()를 사용합니다.

```
desc_stat_long %>%
  pivot_wider(names_from = c(TRT01A, SEX), values_from = values)

# A tibble: 5 x 7
  category Placebo_Female Placebo_Male `Xanomeline Low Dose_Female`
  <chr>      <chr>          <chr>          <chr>
1 n          46            33            47
2 mean       76.1        73.4        76.4
3 med        77.5        74            78
4 sd         8.5         8.1         7.6
5 range_minmax (59, 88)    (52, 85)    (56, 87)
# i 3 more variables: `Xanomeline Low Dose_Male` <chr>,
#   `Xanomeline High Dose_Female` <chr>, `Xanomeline High Dose_Male` <chr>
```

8. 최종 레이블 정리.

category의 변수명을 실제 표에 표시될 이름으로 변경합니다. case_when()을 활용합니다.

```
agestat_cat <- desc_stat_long %>%
  pivot_wider(names_from = c(TRT01A, SEX), values_from = values) %>%
  mutate(category = case_when(category == "n" ~ "N",
                              category == "med" ~ "Median",
                              category == "mean" ~ "Mean",
                              category == "sd" ~ "Std Dev",
                              category == "range_minmax" ~ "Range (min, max)"))
```

```
agestat_cat
```

```
# A tibble: 5 x 7
  category      Placebo_Female Placebo_Male `Xanomeline Low Dose_Female`
  <chr>          <chr>          <chr>          <chr>
1 N              46              33              47
2 Mean           76.1           73.4           76.4
3 Median         77.5           74              78
4 Std Dev        8.5            8.1            7.6
5 Range (min, max) (59, 88)      (52, 85)      (56, 87)
# i 3 more variables: `Xanomeline Low Dose_Male` <chr>,
#   `Xanomeline High Dose_Female` <chr>, `Xanomeline High Dose_Male` <chr>
```

5.1 챌린지 1: 체중(Weight) 및 신장(Height) 분석

위 과정을 반복하여 다른 연속형 변수에 대한 요약표를 만들어 보세요.

5.1.1 보너스 팁: 함수(Function) 작성

동일한 작업을 반복해야 한다면 함수를 작성하는 것이 좋습니다.

```
varSummary <- function(.data, variable) {
  .data %>%
    group_by(TRT01AN, TRT01A, SEX) %>%
    summarize(
      mean = mean({{variable}}) %>% round(digits = 1) %>% format(nsmall = 1),
      sd = sd({{variable}}) %>% round(digits = 1) %>% format(nsmall = 1),
      med = median({{variable}}) %>% format(nsmall = 0),
      min = min({{variable}}) %>% format(nsmall = 0),
      max = max({{variable}}) %>% format(nsmall = 0),
      n = n(),
      .groups = 'drop'
    )
}

adsl_eff %>%
  varSummary(AGE)
```

```
# A tibble: 6 x 9
  TRT01AN TRT01A      SEX  mean  sd  med  min  max    n
  <dbl> <chr>      <chr> <chr> <chr> <chr> <chr> <chr> <int>
1      0 Placebo    Female 76.1  8.5  77.5  59   88    46
2      0 Placebo    Male   73.4  8.1  74    52   85    33
3     54 Xanomeline Low Dose Female 76.4  7.6  78    56   87    47
4     54 Xanomeline Low Dose Male   75.6  8.7  77.5  51   88    34
5     81 Xanomeline High Dose Female 74.3  7.5  76    56   88    35
6     81 Xanomeline High Dose Male   73.6  8.3  75    56   86    39
```

참고: 함수 내부에서 데이터셋의 변수를 참조할 때는 { } (Embrace 연산자)를 사용해야 R이 해당 변수를 인식할 수 있습니다.

Chapter 6

요약표 데이터 통합

이 프로젝트에서는 이전 프로젝트(2, 3)에서 작성한 코드를 결합하여 인구통계 요약표(Demography Summary Table)의 최종 데이터를 구성하는 방법을 배웁니다.

6.1 목표

- 빈도와 백분율 데이터 산출
- 기술 통계량 데이터 산출
- 행 기반 결합(bind_rows)을 통한 최종 데이터셋 생성

6.2 데이터 소스

익명화된 CDISC 데이터셋: <https://github.com/phuse-org/phuse-scripts/tree/master/data/adam/cdisc>

Chapter 7

미니 프로젝트 시작

1. ADSL 데이터 로드 및 전처리

```
library(tidyverse)
```

Warning: package 'ggplot2' was built under R version 4.4.3

Warning: package 'tibble' was built under R version 4.4.3

Warning: package 'purrr' was built under R version 4.4.2

Warning: package 'lubridate' was built under R version 4.4.2

-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --

v dplyr 1.1.4 v readr 2.1.5

v forcats 1.0.0 v stringr 1.5.1

v ggplot2 3.5.2 v tibble 3.3.0

v lubridate 1.9.4 v tidyr 1.3.1

v purrr 1.0.4

-- Conflicts ----- tidyverse_conflicts() --

x dplyr::filter() masks stats::filter()

x dplyr::lag() masks stats::lag()

i Use the conflicted package (<<http://conflicted.r-lib.org/>>) to force all conflicts to become errors

```
library(rio)
```

```
adsl <- import(file = "./data/adsl.xpt")
```

```
adsl_eff <- adsl %>%
```

```
  filter(EFFFL == "Y" ) %>%
```

```
  mutate(SEX = recode(SEX, "M" = "Male", "F" = "Female"))
```

2. 치료군 및 성별별 빈도(n)와 총계(N) 산출

```
Big_N_cnt <- adsl_eff %>%
```

```
  group_by(TRT01AN, TRT01A) %>%
```

```
  count(name = "N")
```

```
small_n_cnt <- adsl_eff %>%
```

```
  group_by(TRT01AN, TRT01A, SEX) %>%
```

```
  count(name = "n")
```

7.0.1 여담: 빈도가 0인 범주 처리하기

데이터에 특정 범주의 관측치가 없는 경우, 해당 행이 누락될 수 있습니다. 이를 방지하기 위해 `complete()` 함수를 사용하여 모든 가능한 조합을 생성하고 빈도를 0으로 채울 수 있습니다.

```
myData <- tribble(
  ~TRT01AN, ~TRT01A, ~SEX,
  1, "Placebo", "M",
  1, "Placebo", "F",
  2, "Active", "F",
  2, "Active", "F",
  3, "Comparator", "M",
  3, "Comparator", "M"
)

# complete (Active-Male, Comparator-Female) 0
myData %>%
  group_by(TRT01AN, TRT01A, SEX) %>%
  count(name = "n") %>%
  ungroup() %>%
  complete(nesting(TRT01AN, TRT01A), SEX, fill = list(n = 0))
```

```
# A tibble: 6 x 4
  TRT01AN TRT01A     SEX      n
  <dbl> <chr>    <chr> <int>
1      1 Placebo    F      1
2      1 Placebo    M      1
3      2 Active     F      2
4      2 Active     M      0
5      3 Comparator F      0
6      3 Comparator M      2
```

3. 연령 그룹(Age Group) 빈도 산출 및 병합

```
Agegrp_N_cnt <- adsl_eff %>%
  group_by(TRT01AN, TRT01A, AGEGR1) %>%
  count(name = "age_total")

age_n_cnt <- adsl_eff %>%
  group_by(TRT01AN, TRT01A, SEX, AGEGR1) %>%
  count(name = "age_n")

age_mrg_cnt <- age_n_cnt %>%
  left_join(Agegrp_N_cnt, by = c("TRT01AN", "TRT01A", "AGEGR1")) %>%
  left_join(Big_N_cnt, by = c("TRT01AN", "TRT01A")) %>%
  left_join(small_n_cnt, by = c("TRT01A", "TRT01AN", "SEX")) %>%
  ungroup()
```

4. 백분율 계산 및 포맷 지정

```
age_n_pct <- age_mrg_cnt %>%
  mutate(perc_age = round((age_n/n)*100, 1)) %>%
  mutate(perc_achar = format(perc_age, nsmall = 1)) %>%
  mutate(npct = paste(age_n, paste0("(", perc_achar, ")"))) %>%
  select(AGEGR1, TRT01A, SEX, npct)
```

5. 데이터 전치 및 레이블 변경

```
age_cat <- age_n_pct %>%
  pivot_wider(names_from = c(TRT01A, SEX),
```



```

      values_from = npct,
      values_fill = "0",
      names_sep = "_" %>%
rename(category = AGEGR1) %>%
arrange(category)

```

7.0.2 여담: 팩터(Factor)를 이용한 정렬 순서 제어

R은 문자형 변수를 기본적으로 영숫자 순으로 정렬합니다. 연령 그룹처럼 논리적 순서가 있는 경우 `factor()`를 사용하여 순서를 명시적으로 정의하는 것이 좋습니다.

```

myData <- tibble::tribble(
  ~ID, ~age,
  1, "18-44",
  2, ">=65",
  3, "45-64")

#
myData <- myData %>%
  mutate(age = factor(age, levels = c("18-44", "45-64", ">=65")))

myData %>% arrange(age)

# A tibble: 3 x 2
   ID age
<dbl> <fct>
1     1 18-44
2     3 45-64
3     2 >=65

```

6. 연령 기술 통계량 산출 (프로젝트 3 복습)

```

age_stat <- adsl_eff %>%
  group_by(TRT01A, SEX) %>%
  summarize(mean = mean(AGE) %>% round(1) %>% format(nsmall = 1),
            sd = sd(AGE) %>% round(1) %>% format(nsmall = 1),
            med = median(AGE) %>% round(1) %>% format(nsmall = 1),
            min = min(AGE) %>% format(nsmall = 1),
            max = max(AGE) %>% format(nsmall = 1),
            n = n() %>% as.character(),
            .groups = 'drop') %>%
  mutate(range_minmax = paste0("(", min, ", ", max, ")"))

```

7. 기술 통계량 데이터 전치

```

agestat_cat <- age_stat %>%
  select(TRT01A, SEX, n, mean, med, sd, range_minmax) %>%
  mutate(across(where(is.numeric), as.character)) %>%
  pivot_longer(-c(TRT01A, SEX), names_to = "category", values_to = "values") %>%
  pivot_wider(names_from = c(TRT01A, SEX), values_from = values) %>%
  mutate(category = case_when(category == "n" ~ "N",
                              category == "med" ~ "Median",
                              category == "mean" ~ "Mean",
                              category == "sd" ~ "Std Dev",
                              category == "range_minmax" ~ "Range (min, max)"))

```

8. 최종 데이터 결합

`bind_rows()` 함수를 사용하여 위에서 만든 범주형 빈도 데이터(`age_cat`)와 연속형 요약 통계 데이터(`agestat_cat`)를 위아래로 합칩니다. 이는 SAS의 SET 문과 유사한 역할을 합니다.

```
dm_allcomb <- bind_rows(age_cat, agestat_cat)
dm_allcomb
```

```
# A tibble: 8 x 7
  category      Placebo_Female Placebo_Male `Xanomeline Low Dose_Female`
  <chr>          <chr>          <chr>          <chr>
1 65-80          20 (43.5)        20 (60.6)      26 (55.3)
2 <65            8 (17.4)         5 (15.2)       4 ( 8.5)
3 >80           18 (39.1)        8 (24.2)      17 (36.2)
4 N              46              33             47
5 Mean           76.1           73.4           76.4
6 Median          77.5           74.0           78.0
7 Std Dev         8.5            8.1            7.6
8 Range (min, max) (59.0, 88.0) (52.0, 85.0) (56.0, 87.0)
# i 3 more variables: `Xanomeline Low Dose_Male` <chr>,
#   `Xanomeline High Dose_Female` <chr>, `Xanomeline High Dose_Male` <chr>
```

7.1 챌린지

1. 연령 범주의 표시 순서를 적절하게 조정해 보세요. (<65, 65-80, >80)
2. N (전체 대상자 수) 행을 연령 범주 행보다 위로 이동시켜 보세요.
3. 민족(Ethnicity)과 인종(Race) 변수에 대해서도 동일한 과정을 수행하여 요약표를 확장해 보세요.

Chapter 8

{ggplot2}를 이용한 시각화 기초

이 미니 프로젝트에서는 R의 대표적인 시각화 패키지인 {ggplot2}를 사용하여 데이터를 시각화하는 방법을 배웁니다. {ggplot2}는 레이어(Layer) 구조를 기반으로 하며, 수많은 확장 패키지를 통해 기능을 무한히 확장할 수 있다는 장점이 있습니다.

8.1 목표

- ggplot() 함수와 aes() (Aesthetic mapping) 이해
- geom_point(), geom_line()을 이용한 산점도 및 선 그래프 생성
- labs()를 이용한 제목 및 축 레이블 설정
- facet_wrap()을 이용한 그룹별 분할 그래프 생성

8.2 데이터 소스

익명화된 CDISC 데이터셋: <https://github.com/phuse-org/phuse-scripts/tree/master/data/adam/cdisc>

8.3 1. 데이터셋 준비

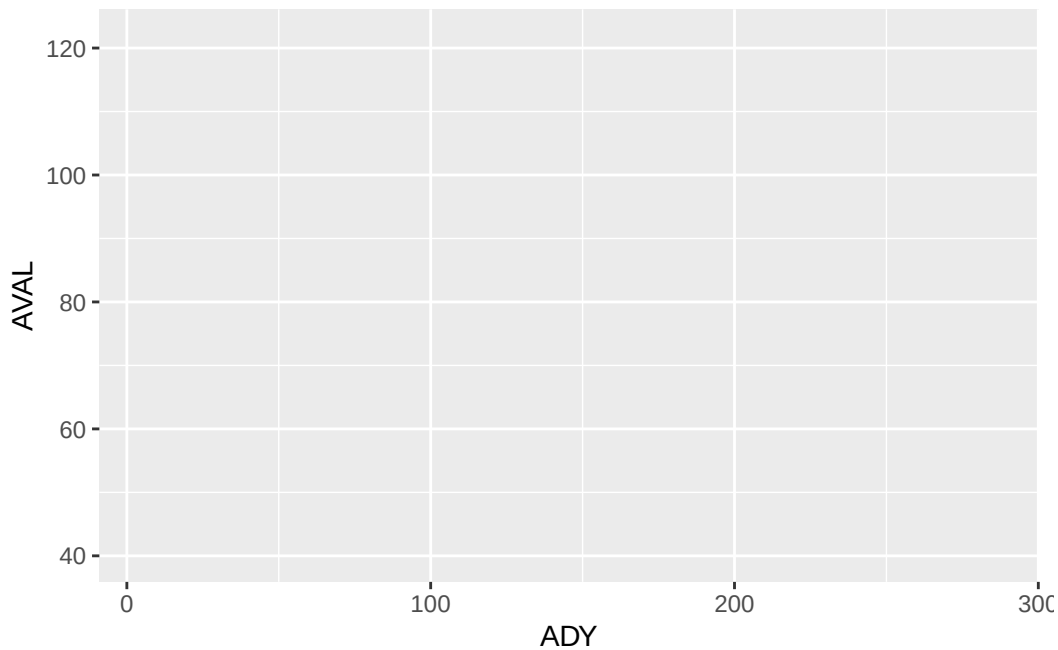
CDISC의 분석 활력 징후(ADVS) 데이터를 사용합니다. 여기서는 심박수(Pulse Rate) 측정치에 집중하며, 치료 시작 후 경과 주차를 나타내는 StudyWeek 변수를 새로 계산합니다.

```
ADVS <- import(file = "./data/adv.s.xpt")
PR <- ADVS %>%
  filter(PARAMN == 3) %>% #
  filter(VISITNUM > 3) %>% #
  mutate(StudyWeek = floor(ADY/7))
```

8.4 2. ggplot 기본 구조

ggplot() 함수는 그래프를 그릴 빈 “캔버스”를 생성합니다. data 인수는 사용할 데이터셋을, mapping 인수는 데이터의 열을 그래프의 시각적 요소(축, 색상 등)에 어떻게 연결할지(aes() 함수 사용) 정의합니다.

```
# x      (ADY), y      (AVAL)
ggplot(data = PR, mapping = aes(x = ADY, y = AVAL))
```



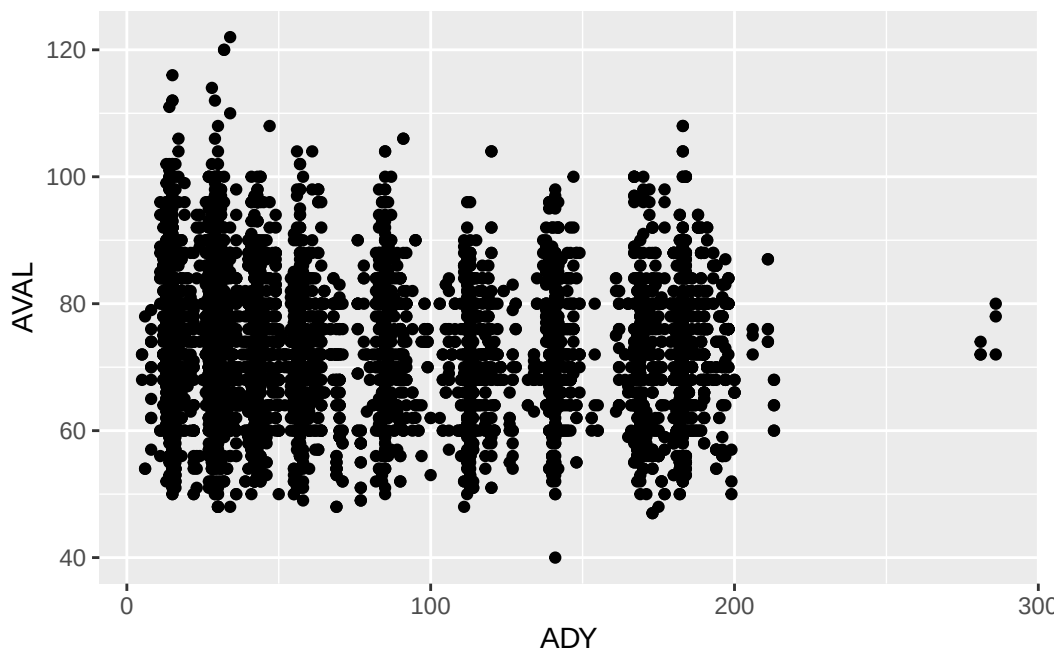
팁: 학습 초기에는 `mapping = aes(...)`와 같이 인수를 명시적으로 작성하는 것을 권장합니다.

8.5 3. 기하학적 객체(Geom) 추가

산점도를 그리려면 위 문구에 `+ geom_point()`를 추가합니다. `+` 연산자는 레이어를 쌓는 역할을 하며, 반드시 줄 끝에 위치해야 합니다.

```
ggplot(data = PR, mapping = aes(x = ADY, y = AVAL)) +  
  geom_point()
```

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

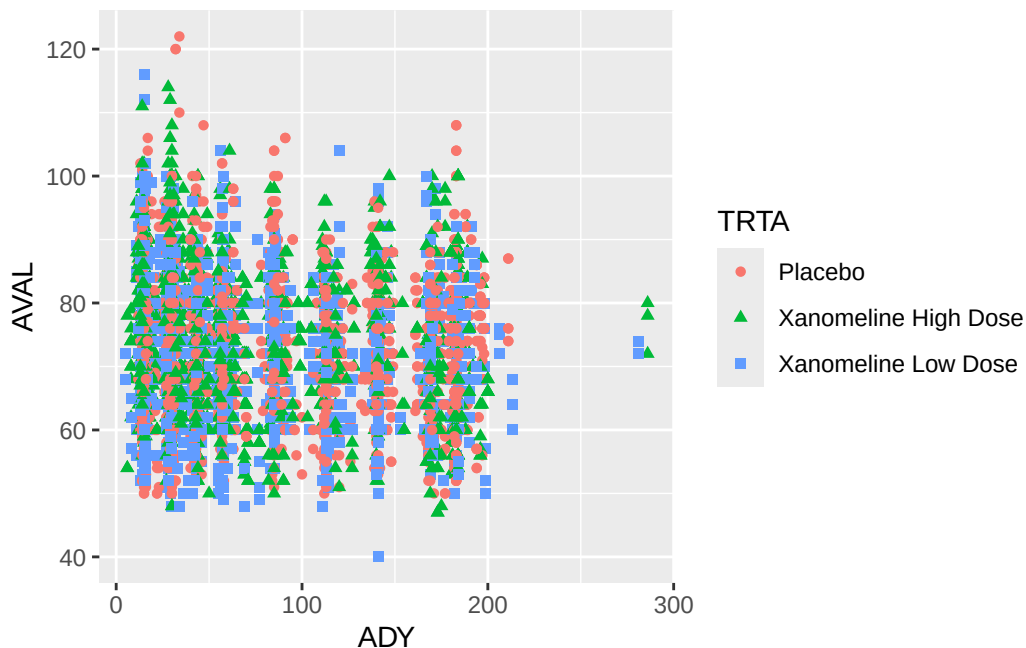


8.6 4. 데이터 그룹화

치료군(TRTA)별로 데이터를 구분해 보겠습니다. `aes()` 내부에 `group`, `colour`, `shape` 등을 지정할 수 있습니다.

```
ggplot(data = PR, mapping = aes(x = ADY, y = AVAL,
                                colour = TRTA, shape = TRTA)) +
  geom_point()
```

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

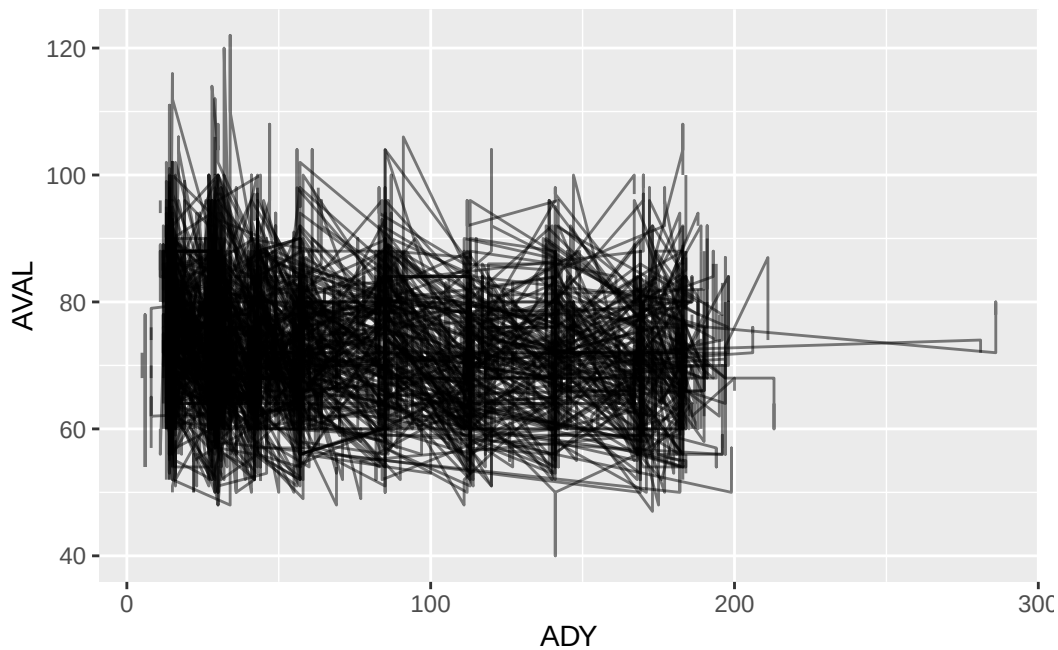


8.7 5. 개별 피험자별 선 연결 (스파게티 플롯)

동일한 피험자의 측정치를 선으로 연결하여 변화 추이를 확인하는 “스파게티 플롯 (Spaghetti Plot)”을 그려봅니다. 피험자 식별자인 `USUBJID`를 `group`에 매핑합니다.

```
plot1 <- PR %>%
  ggplot(mapping = aes(x = ADY, y = AVAL, group = USUBJID)) +
  geom_line(alpha = 0.5) #
plot1
```

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_line()``).



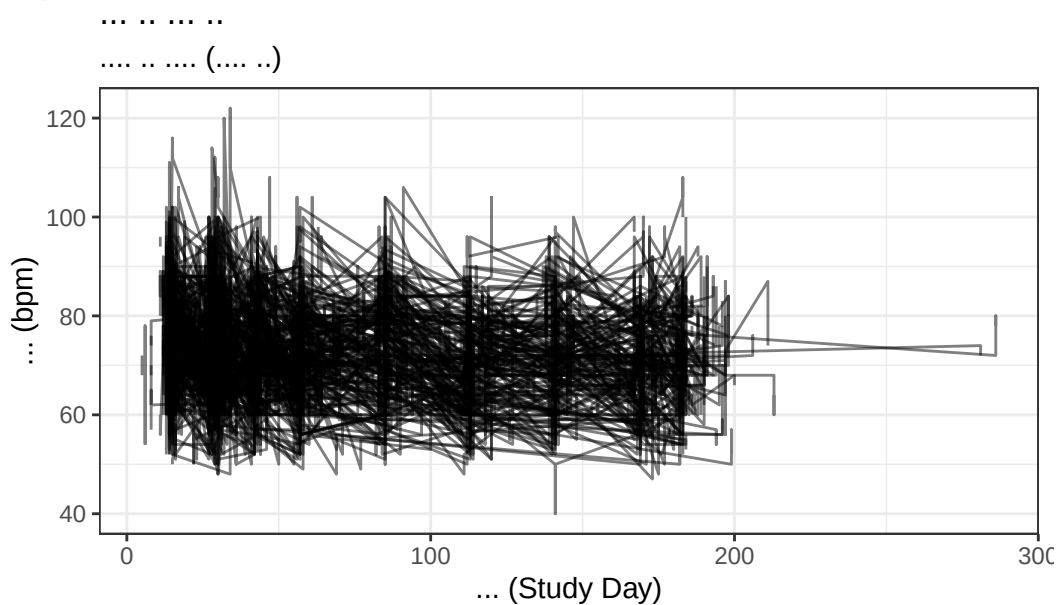
8.8 6. 레이블 및 테마 설정

`labs()` 함수로 제목과 축 레이블을 한글로 수정하고, `theme_bw()`를 적용하여 깔끔한 테마로 변경합니다.

```
plot2 <- plot1 +
  labs(title = " ",
        subtitle = " ( )",
        x = " (Study Day)",
        y = " (bpm)",
        caption = paste(" :", Sys.Date()))

plot3 <- plot2 + theme_bw()
plot3
```

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_line()``).



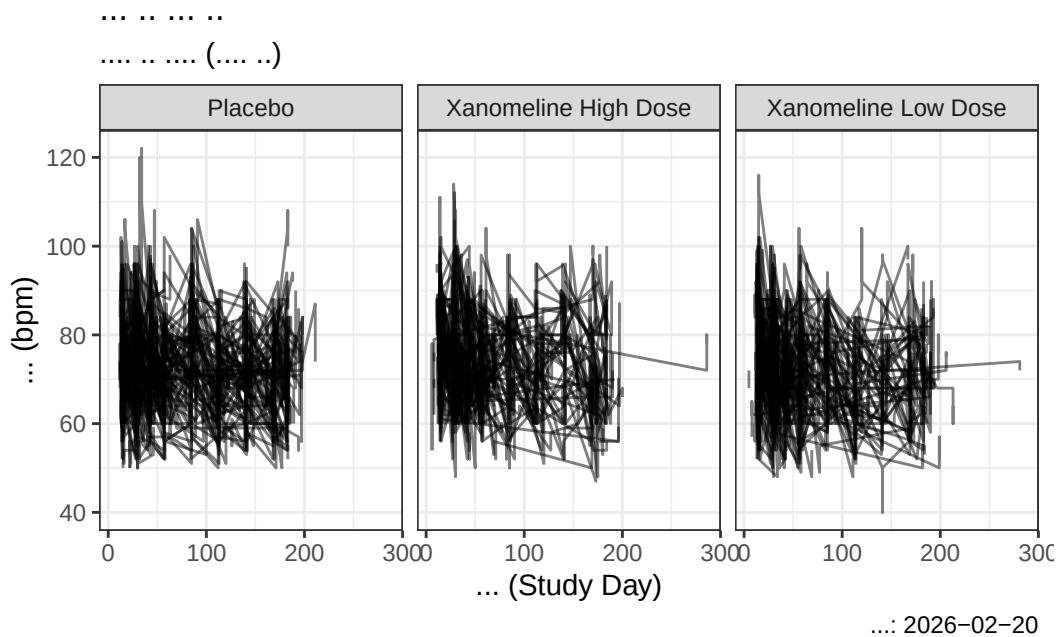
...: 2026-02-20

8.9 7. 패싯(Facet)을 이용한 그룹별 분할

치료군(TRTA)별로 그래프를 나누어 보고 싶을 때 `facet_wrap()`을 사용합니다.

```
plot4 <- plot3 +
  facet_wrap(~ TRTA)
plot4
```

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_line()``).



8.10 8. 결과 저장

`ggsave()`를 사용하여 결과물을 이미지 파일로 저장할 수 있습니다.

```
# ggsave("heart_rate_plot.png", plot = plot4, width = 8, height = 6, units = "in")
```

8.11 9. 요약 통계(중앙값) 추가

개별 데이터 위에 전체 중앙값의 흐름을 겹쳐 그릴 수 있습니다. `stat_summary()`를 사용합니다.

```
#
dataWeeks <- PR %>%
  filter(StudyWeek %in% c(0, 5, 10, 15, 20, 25, 30)) %>%
  mutate(ADY = StudyWeek * 7)

plot3 +
  stat_summary(data = dataWeeks,
    mapping = aes(x = ADY, group = NULL),
    fun = median,
    geom = "line",
    colour = "red",
    linewidth = 1)
```

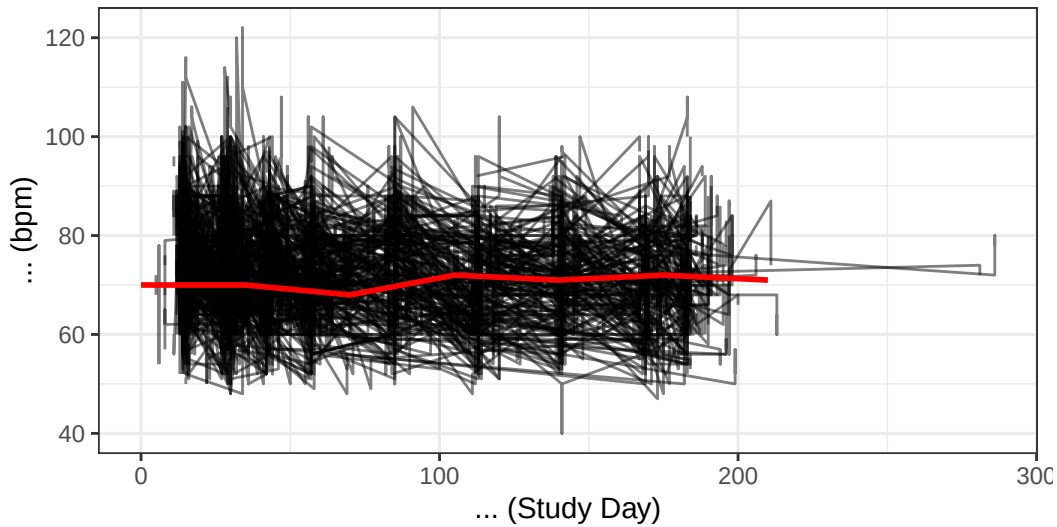
Warning: Removed 1 row containing non-finite outside the scale range (``stat_summary()``).

Warning: Removed 1 row containing missing values or values outside the scale range

```
(`geom_line()`).
```

```
... ..
```

```
.... .. (.... ..)
```



...: 2026-02-20

8.12 10. 축 범위 조정 (Zooming)

데이터를 삭제하지 않고 특정 영역을 확대해서 보려면 `coord_cartesian()`을 사용합니다.

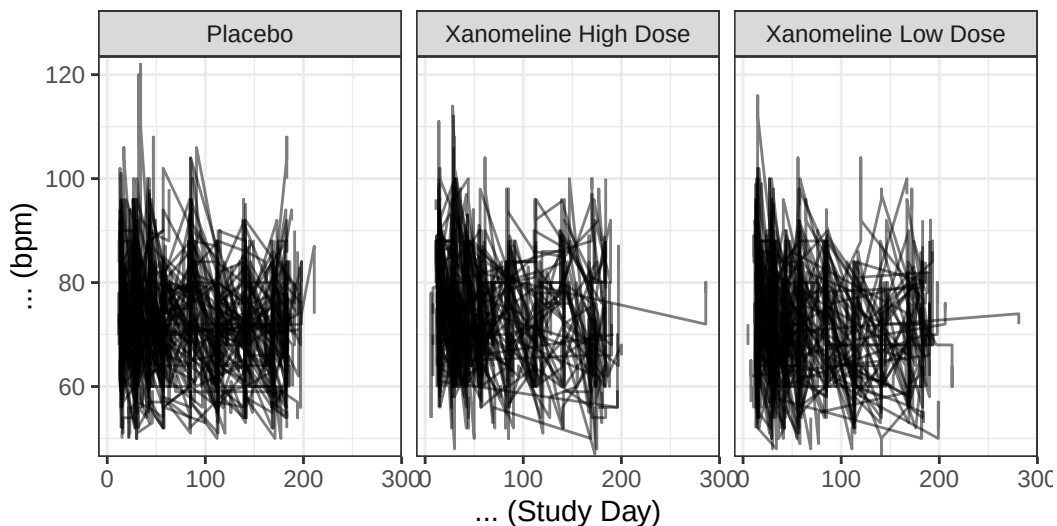
```
plot4 +  
  coord_cartesian(ylim = c(50, 120))
```

Warning: Removed 1 row containing missing values or values outside the scale range

```
(`geom_line()`).
```

```
... ..
```

```
.... .. (.... ..)
```



...: 2026-02-20

8.13 챌린지

수축기 혈압(Systolic BP), 이완기 혈압(Diastolic BP), 체온(Temperature) 등에 대해서도 동일한 과정을 수행해 보세요. 데이터의 특성에 맞게 축 레이블을 수정하는 것을 잊지 마세요.

Chapter 9

{ggplot2} 확장 패키지 활용

이 연습에서는 {ggplot2}의 기능을 보강해주는 5가지 확장 패키지를 소개합니다: {gghighlight}, {ggdist}, {ggridges}, {patchwork}, {cowplot}

9.1 목표

- {gghighlight}를 이용한 특정 데이터 강조
- {ggdist} 및 {gggridges}를 이용한 분포 시각화
- {patchwork}를 이용한 여러 그래프 배치
- {cowplot}을 이용한 주석 및 워터마크 추가

9.2 데이터 소스

익명화된 CDISC 실험실 검사(ADLBC) 데이터셋: <https://github.com/phuse-org/phuse-scripts/tree/master/data/adam/cdisc>

9.3 1. 데이터셋 준비

알라닌 아미노전이효소(ALT) 결과에 집중하여 분석용 데이터를 준비합니다. 치료 시작 후 주차를 나타내는 WEEK 변수를 생성합니다.

```
ALT <- import("./data/adlbc.xpt") %>%
  filter(PARAMCD == "ALT")

ALT2 <- ALT %>%
  filter(VISITNUM > 3) %>%
  mutate(WEEK = floor(ADY/7))

# (Factor)
TREATfac <- ALT2 %>%
  select(TRTA, TRTAN) %>%
  distinct() %>%
  arrange(TRTAN)

ALT2 <- ALT2 %>%
  mutate(TREATTXT = factor(TRTA, levels = TREATfac$TRTA))

#
uniqueVal <- function(x){
  res <- unique(x)
```

```

if(length(res) > 1) warning("                .")
return(res)
}

```

9.4 2. ALT 변화 추이 스파게티 플롯

피험자별 ALT 수치 변화를 선으로 연결하고, 정상 범위(Normal Range)를 음영(geom_ribbon)으로 표시합니다.

```

ALTmin <- uniqueVal(ALT2$A1LO)[1]
ALTmax <- uniqueVal(ALT2$A1HI)[1]

```

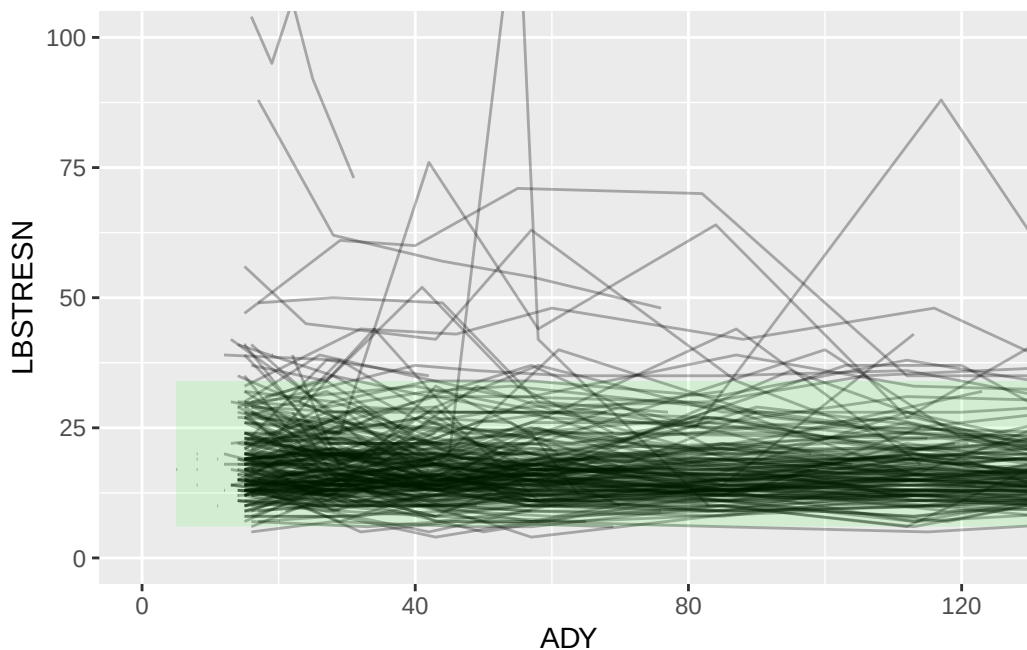
Warning in uniqueVal(ALT2\$A1HI):

```

plot1 <- ALT2 %>%
  ggplot(mapping = aes(x = ADY, y = LBSTRESN)) +
  geom_line(mapping = aes(group = USUBJID), alpha = 0.3) +
  geom_ribbon(aes(ymin = ALTmin, ymax = ALTmax), fill = "green", alpha = 0.1) +
  coord_cartesian(ylim = c(0, 100), xlim = c(0, 125))

```

plot1



9.5 3. {gghighlight}로 이상치 강조

정상 범위를 벗어난(“HIGH” 또는 “LOW”) 기록이 한 번이라도 있는 피험자의 라인만 강조해 보겠습니다.

```
library(gghighlight)
```

Warning: package 'gghighlight' was built under R version 4.4.3

```

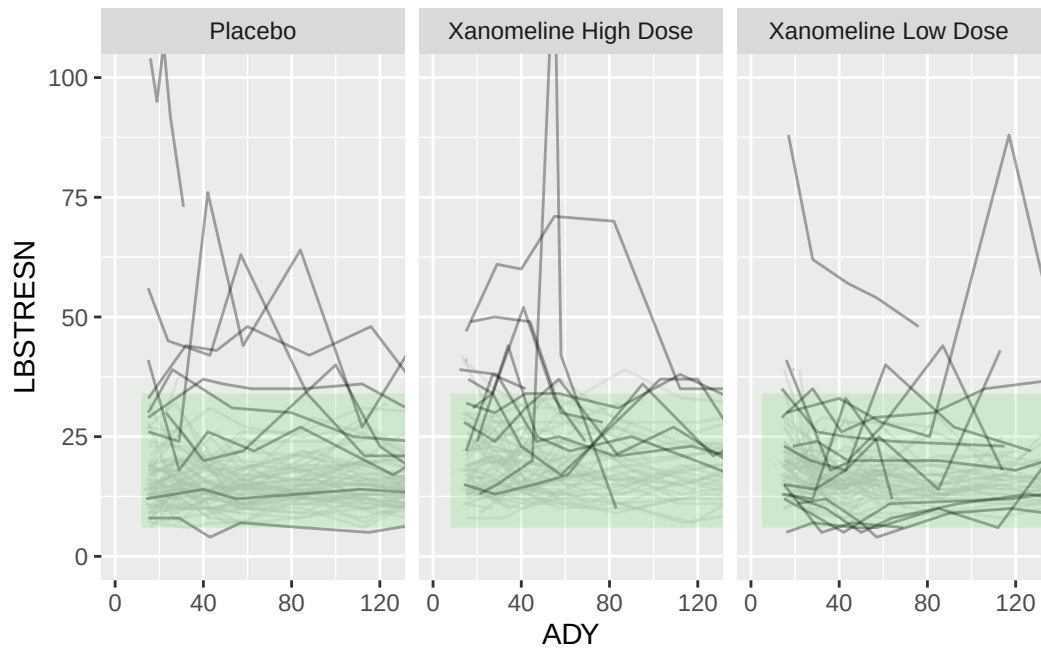
plot1c <- plot1 +
  gghighlight(any(LBNRIND %in% c("HIGH", "LOW")), calculate_per_facet = TRUE) +
  facet_wrap(~ TRTA)

```

label_key: USUBJID

Too many data series, skip labeling

plot1c

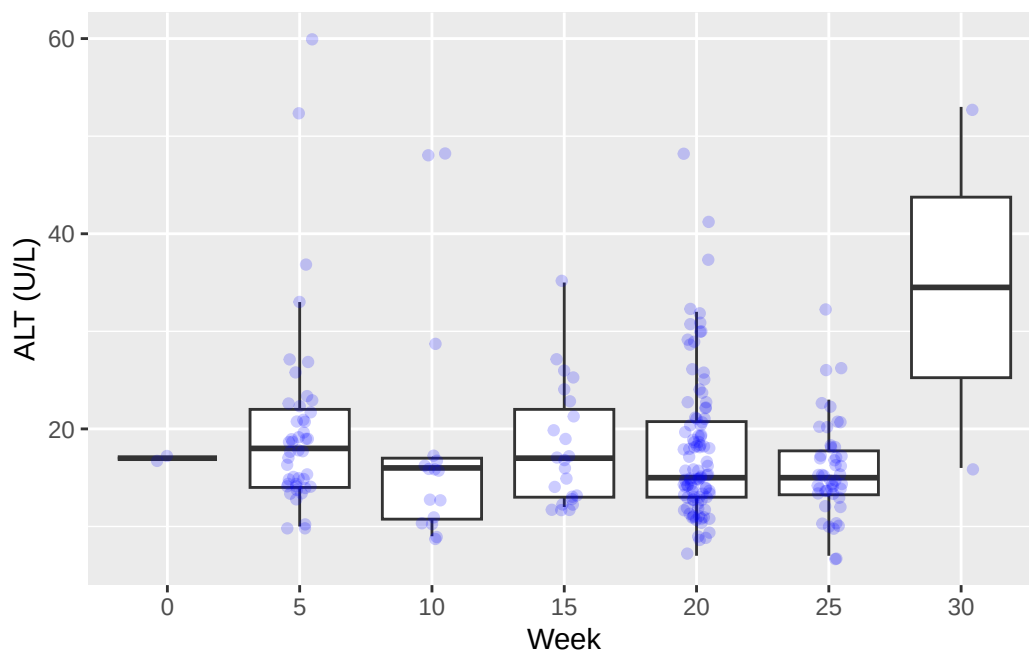


9.6 4. 분포 시각화 (Boxplot & Jitter)

상자 그림(Boxplot)은 데이터의 분포를 요약해 주지만 데이터의 개수는 보여주지 못합니다. `geom_jitter`를 겹쳐서 보완해 봅니다.

```
plot2b <- ALT2 %>%
  filter(WEEK %in% c(0, 5, 10, 15, 20, 25, 30)) %>%
  ggplot(mapping = aes(x = as.factor(WEEK), y = LBSTRESN)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(alpha = 0.2, width = 0.1, color = "blue") +
  labs(x = "Week", y = "ALT (U/L)")
```

plot2b



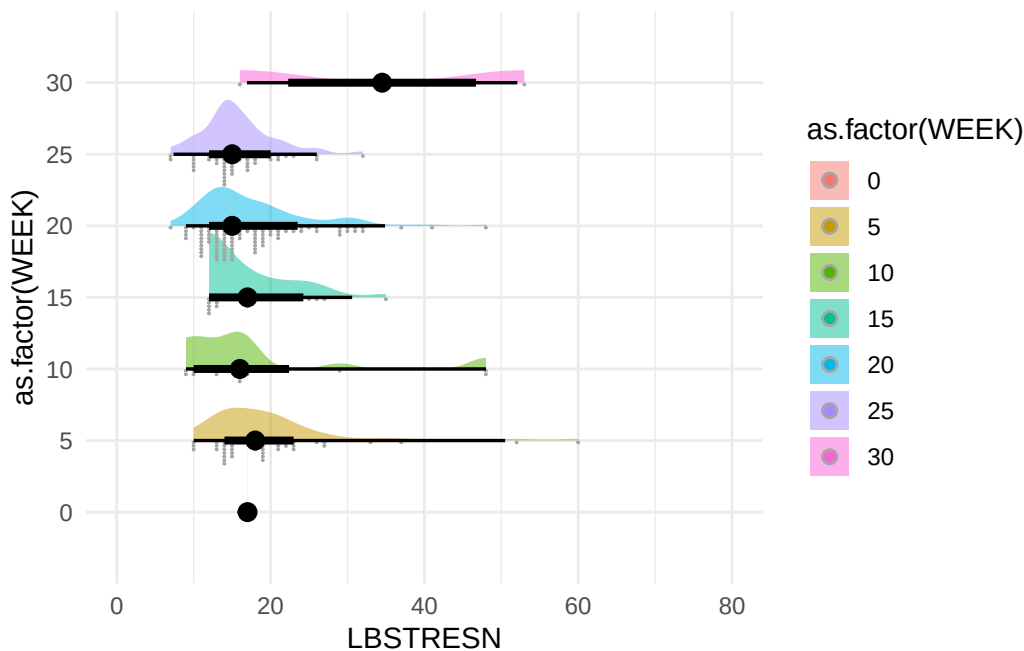
9.7 5. 분포 시각화 (Dotplot & Ridge plot)

{ggdist}를 사용하면 점의 밀도를 직접적으로 보여주는 도트 플롯(Dotplot)을 그릴 수 있습니다.

```
library(ggdist)
```

Warning: package 'ggdist' was built under R version 4.4.3

```
# Raincloud plot
ALT2 %>%
  filter(WEEK %in% c(0, 5, 10, 15, 20, 25, 30)) %>%
  ggplot(mapping = aes(x = LBSTRESN, y = as.factor(WEEK), fill = as.factor(WEEK))) +
  stat_slab(alpha = 0.5) +
  stat_dotsinterval(side = "bottom", scale = 0.5) +
  coord_cartesian(xlim = c(0, 80)) +
  theme_minimal()
```



{ggribes}를 이용한 능선 플롯(Ridge Plot)은 여러 그룹의 분포를 비교할 때 매우 효율적입니다.

```
library(ggribes)
```

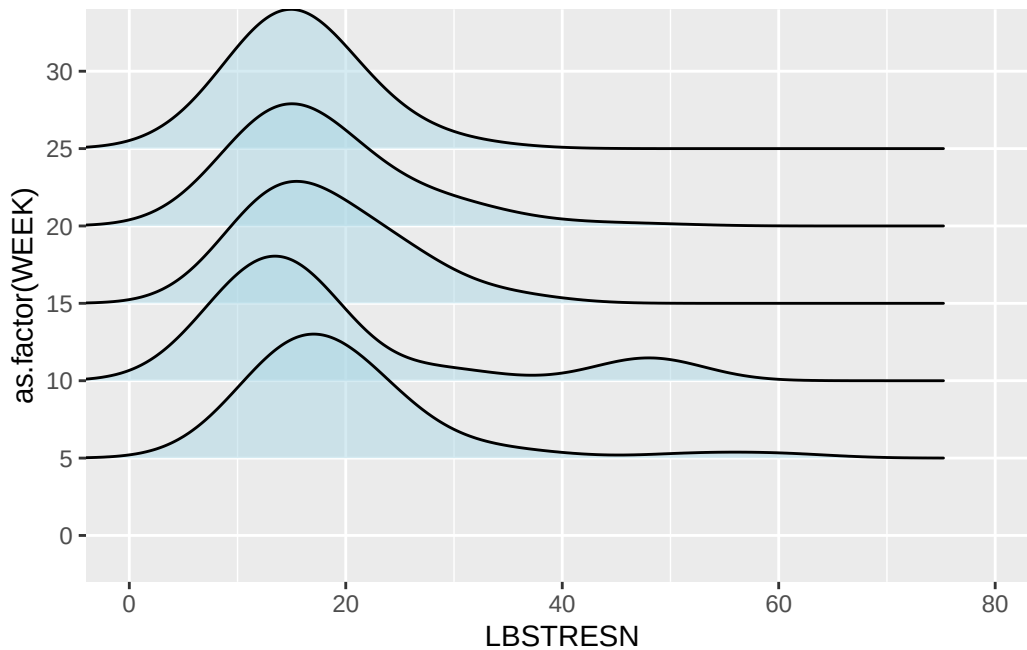
Attaching package: 'ggribes'

The following objects are masked from 'package:ggdist':

```
scale_point_color_continuous, scale_point_color_discrete,
scale_point_colour_continuous, scale_point_colour_discrete,
scale_point_fill_continuous, scale_point_fill_discrete,
scale_point_size_continuous
```

```
ALT2 %>%
  filter(WEEK %in% c(0, 5, 10, 15, 20, 25, 30)) %>%
  ggplot(mapping = aes(x = LBSTRESN, y = as.factor(WEEK))) +
  geom_density_ridges(alpha = 0.5, fill = "lightblue") +
  coord_cartesian(xlim = c(0, 80))
```

Picking joint bandwidth of 5.07



9.8 6. {patchwork}를 이용한 그래프 배치

여러 개의 그래프를 하나로 합쳐 보겠습니다. l는 좌우 배치를, /는 상하 배치를 의미합니다.

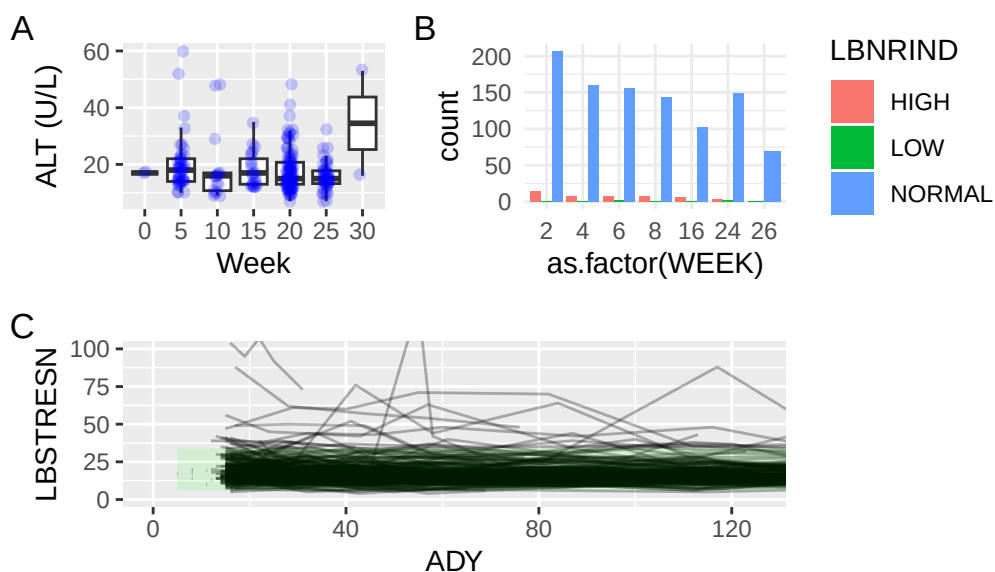
```
library(patchwork)
```

Warning: package 'patchwork' was built under R version 4.4.3

```
#
plot3 <- ALT2 %>%
  filter(WEEK %in% c(2, 4, 6, 8, 16, 24, 26)) %>%
  ggplot(aes(x = as.factor(WEEK), fill = LBNRIND)) +
  geom_bar(position = "dodge") +
  theme_minimal()

combined_plot <- (plot2b | plot3) / plot1
combined_plot + plot_annotation(title = "ALT", tag_levels = "A")
```

ALT



9.9 7. {cowplot}으로 워터마크 추가

{cowplot}은 그래프 위에 텍스트를 그리거나 워터마크를 넣는 작업을 쉽게 만들어줍니다.

```
library(cowplot)
```

Warning: package 'cowplot' was built under R version 4.4.3

Attaching package: 'cowplot'

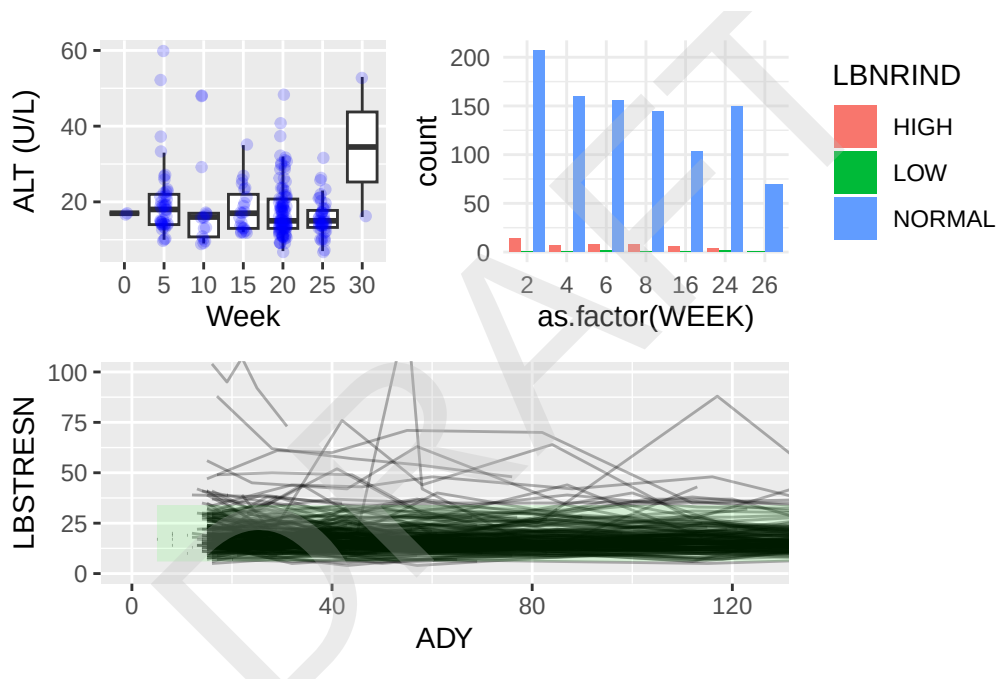
The following object is masked from 'package:patchwork':

```
align_plots
```

The following object is masked from 'package:lubridate':

```
stamp
```

```
ggdraw(combined_plot) +  
  draw_label("DRAFT", color = "grey", alpha = 0.3, size = 100, angle = 45)
```



9.10 챌린지

지금까지 배운 내용을 활용하여 시간에 따른 ALT 수치 변화 그래프를 만들어 보세요. 특히 정상 범위를 벗어난 데이터 포인트를 빨간색으로 표시하고, 치료군별로 패킷을 나누어 보세요.

Chapter 10

함수 작성 기초

R에서 함수를 작성하는 방법과 문제가 발생했을 때 디버깅하는 방법을 배웁니다. **경험상 동일한 코드를 두 번 이상 복사해서 사용한다면, 함수로 만드는 것이 좋습니다.** R 패키지 또한 특정 목적을 위해 만들어진 함수, 문서, 테스트의 모음입니다.

10.1 1. 소개

함수는 SAS의 매크로(Macro)와 유사한 역할을 합니다. 처음에는 단순한 코드로 시작하여 입력을 식별하고 작업을 결정한 뒤, 점차 인수를 추가하여 사용자가 설정을 바꿀 수 있도록 고도화해 나갑니다.

미니 프로젝트 2에서 사용했던 치료군별 성별 빈도 산출 코드를 예로 들어보겠습니다.

```
library(rio)
library(tidyverse)
```

```
Warning: package 'ggplot2' was built under R version 4.4.3
```

```
Warning: package 'tibble' was built under R version 4.4.3
```

```
Warning: package 'purrr' was built under R version 4.4.2
```

```
Warning: package 'lubridate' was built under R version 4.4.2
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
v dplyr      1.1.4      v readr      2.1.5
```

```
v forcats    1.0.0      v stringr    1.5.1
```

```
v ggplot2    3.5.2      v tibble     3.3.0
```

```
v lubridate  1.9.4      v tidyr      1.3.1
```

```
v purrr      1.0.4
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
adsl_saf <- import("./data/adsl.xpt") %>%
  filter(SAFFL == "Y")
```

```
#      (N)
```

```
Big_N_cnt <- adsl_saf %>%
```

```
  group_by(TRT01AN, TRT01A) %>%
```

```
  count(name = "N")
```

```
#      (n)
```

```
adsl_mrg_cnt <- adsl_saf %>%
```

```
  group_by(TRT01AN, TRT01A, SEX) %>%
```

```
  count(name = "n") %>%
```

```

left_join(Big_N_cnt, by = c("TRT01A", "TRT01AN")) %>%
mutate(perc = round((n/N)*100, 1)) %>%
mutate(npct = paste0(n, " (", format(perc, nsmall = 1), ")")) %>%
mutate(SEX = recode(SEX, "M" = "Male", "F" = "Female")) %>%
ungroup() %>%
select(TRT01A, SEX, npct) %>%
pivot_wider(names_from = TRT01A, values_from = npct)

adsl_mrg_cnt

```

```

# A tibble: 2 x 4
  SEX      Placebo `Xanomeline Low Dose` `Xanomeline High Dose`
  <chr>   <chr>      <chr>                        <chr>
1 Female 53 (61.6) 50 (59.5)                    40 (47.6)
2 Male   33 (38.4) 34 (40.5)                    44 (52.4)

```

10.2 2. 함수의 구조

함수는 다음의 핵심 요소를 가집니다.

- **함수 이름:** 수행하는 작업을 잘 설명하는 동사(예: calculate_counts)를 권장합니다.
- **인수(Arguments):** 함수에 전달되는 입력값입니다.
- **함수 본문(Body):** { } 내부에 작성되는 실제 실행 코드입니다.
- **반환 값(Return value):** 함수의 실행 결과로 돌려주는 객체입니다.

참고: 함수 내부에서 생성된 변수나 객체는 함수의 로컬 환경에만 존재하며, 명시적으로 반환하지 않으면 함수 밖에서 사용할 수 없습니다.

10.3 3. 함수 생성 실습

RStudio IDE의 스니펫(Snippet) 기능을 활용하면 편리합니다. 콘솔이나 스크립트 창에서 fun을 입력하고 TAB 키를 눌러보세요.

위의 코드를 기반으로 adsl_counts라는 함수를 만들어 보겠습니다. 이 함수는 데이터 파일 경로를 인수로 받습니다.

```

adsl_counts <- function(dataFile) {
  #
  adsl_saf <- haven::read_xpt(dataFile) %>%
    filter(SAFFL == "Y")

  #
  Big_N_cnt <- adsl_saf %>%
    group_by(TRT01AN, TRT01A) %>%
    count(name = "N")

  #
  adsl_mrg_cnt <- adsl_saf %>%
    group_by(TRT01AN, TRT01A, SEX) %>%
    count(name = "n") %>%
    left_join(Big_N_cnt, by = c("TRT01A", "TRT01AN")) %>%
    mutate(perc = round((n/N)*100, 1)) %>%
    mutate(npct = paste0(n, " (", format(perc, nsmall = 1), ")")) %>%
    mutate(SEX = recode(SEX, "M" = "Male", "F" = "Female")) %>%
    ungroup() %>%
    select(TRT01A, SEX, npct) %>%
    pivot_wider(names_from = TRT01A, values_from = npct)
}

```



```
  return(adsl_mrg_cnt)
}
```

이제 함수를 호출하여 결과를 확인해 봅니다.

```
inFile <- "../data/adsl.xpt"
adsl_counts(inFile)
```

```
# A tibble: 2 x 4
  SEX      Placebo  `Xanomeline Low Dose` `Xanomeline High Dose`
<chr> <chr>      <chr>                <chr>
1 Female 53 (61.6) 50 (59.5)                40 (47.6)
2 Male   33 (38.4) 34 (40.5)                44 (52.4)
```

10.4 4. 방어적 프로그래밍 (Defensive Programming)

사용자가 엉뚱한 데이터를 넣었을 때 에러를 발생시키거나 안내 메시지를 띄우는 것이 좋습니다. 예를 들어 파일명에 “adsl”이 포함되어 있는지 확인하는 코드를 추가할 수 있습니다.

```
# try() stop()
# if(!stringr::str_detect(inFile, "adsl")) stop("ADSL          .")
```

10.5 5. 코드 리팩토링 (Refactoring)

더 효율적이고 읽기 쉬운 코드로 개선하는 과정을 리팩토링이라고 합니다. 현재 함수는 파일을 매번 새로 읽어오는데, 이는 대용량 데이터나 네트워크 환경에서 비효율적입니다. 데이터를 먼저 읽어온 뒤, **데이터 프레임**을 인수로 받도록 함수를 수정해 보겠습니다.

```
#
calculate_adsl_counts <- function(.data) {
  Big_N_cnt <- .data %>%
    group_by(TRT01AN, TRT01A) %>%
    count(name = "N", .groups = 'drop')

  res <- .data %>%
    group_by(TRT01AN, TRT01A, SEX) %>%
    count(name = "n") %>%
    left_join(Big_N_cnt, by = c("TRT01A", "TRT01AN")) %>%
    mutate(npct = paste0(n, " (", format(round((n/N)*100, 1), nsmall = 1), "%)") %>%
    ungroup() %>%
    select(TRT01A, SEX, npct) %>%
    pivot_wider(names_from = TRT01A, values_from = npct)

  return(res)
}
```

```
#
adsl_prepared <- haven::read_xpt("../data/adsl.xpt") %>%
  filter(SAFFL == "Y") %>%
  mutate(SEX = recode(SEX, "M" = "Male", "F" = "Female"))

calculate_adsl_counts(adsl_prepared)
```

```
# A tibble: 2 x 4
  SEX      Placebo  `Xanomeline Low Dose` `Xanomeline High Dose`
<chr> <chr>      <chr>                <chr>
1 Female 53 (61.6) 50 (59.5)                40 (47.6)
```

2 Male 33 (38.4) 34 (40.5) 44 (52.4)

10.6 6. 생략 부호 (Ellipses, ...) 활용

하위 함수에 여러 인수를 유연하게 전달하고 싶을 때 ...을 사용합니다. 예를 들어 반올림 자릿수를 사용자가 결정하게 할 수 있습니다.

```
mySummary <- function(myData, ...){
  myData %>%
    group_by(TRT01AN, TRT01A) %>%
    summarise(mean = round(mean(AGE), ...), .groups = 'drop')
}
```

```
# digits    ...    round
mySummary(adsl_prepared, digits = 2)
```

```
# A tibble: 3 x 3
  TRT01AN TRT01A          mean
  <dbl> <chr>          <dbl>
1      0 Placebo          75.2
2     54 Xanomeline Low Dose 75.7
3     81 Xanomeline High Dose 74.4
```

10.7 챌린지

미니 프로젝트 6에서 작성한 ALT 시각화 코드를 함수로 만들어 보세요. 인수로 데이터셋을 받고, 사용자가 축 레이블을 직접 지정할 수 있도록 구성해 보세요.

Chapter 11

반복 처리를 이용한 함수 적용

데이터의 행, 열 또는 서브셋에 대해 함수를 반복적으로 적용하는 방법을 배웁니다. R의 강점인 벡터화(Vectorization)와 반복문을 효율적으로 사용하는 방법을 살펴봅니다.

11.1 목표

- R의 루프(Loop)와 효율성 이해
- apply 계열 함수 활용
- {tidyverse}의 across() 및 rowwise() 활용
- {purrr} 패키지를 이용한 리스트 기반 반복 처리

11.2 1. 벡터화: 루프가 필수가 아닌 이유

SAS에서는 DATA 단계를 통해 행 단위로 처리하는 방식이 익숙하지만, R은 벡터(Vector) 단위 처리에 최적화되어 있습니다. 예를 들어, 모든 열의 비결측치 개수를 셀 때 R은 벡터 전체를 한 번에 평가합니다.

```
# (SUBJID)
adsl_saf %>%
  summarise(nonmiss = sum(!is.na(SUBJID)))
```

```
nonmiss
1      254
```

R은 is.na() 함수를 벡터의 모든 요소에 동시에 적용하여 TRUE/FALSE 벡터를 반환하고, sum()은 이를 1/0으로 취급하여 합계를 구합니다. 명시적인 루프 없이도 매우 빠르게 처리됩니다.

11.3 2. 명시적 루프 (for loop) 사용하기

모든 열에 대해 같은 작업을 반복해야 한다면 루프를 고려할 수 있습니다. 하지만 R에서 루프를 사용할 때는 결과값을 담을 객체를 미리 생성하는 것이 성능상 매우 중요합니다.

```
varNames <- names(adsl_saf)
counts <- vector("integer", length(varNames)) #

for(i in 1:length(varNames)){
  counts[i] <- sum(!is.na(adsl_saf[[varNames[i]]]))
}

names(counts) <- varNames
head(counts)
```

STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM
254	254	254	254	254	254

이 방식은 SAS 사용자에게 익숙할 수 있지만, 결과가 데이터 프레임이 아닌 벡터로 반환되어 후속 작업(select() 등)이 불편할 수 있습니다.

11.4 3. Apply 계열 함수

기본 R의 apply 함수군은 입력을 받아 각 요소에 함수를 적용합니다.

- apply(): 배열이나 행렬의 행(MARGIN=1) 또는 열(MARGIN=2) 단위 처리
- lapply(): 리스트(List) 요소별 처리 (결과는 리스트)
- sapply(): 결과를 벡터나 행렬로 단순화(Simplify)하여 반환

```
#
sapply(adsl_saf, function(x) sum(!is.na(x))) %>% head()
```

STUDYID	USUBJID	SUBJID	SITEID	SITEGR1	ARM
254	254	254	254	254	254

11.5 4. Tidy 반복 처리 (across & rowwise)

{tidyverse}는 더 직관적이고 예측 가능한 반복 처리 방법을 제공합니다.

11.5.1 across()를 이용한 열 단위 처리

summarise()나 mutate() 내부에서 여러 열에 동시에 함수를 적용할 때 사용합니다.

```
#
adsl_saf %>%
  summarise(across(where(is.numeric), median, na.rm = TRUE))
```

Warning: There was 1 warning in `summarise()`.
 i In argument: `across(where(is.numeric), median, na.rm = TRUE)`.
 Caused by warning:
 ! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
 Supply arguments directly to `.fns` through an anonymous function instead.

```
# Previously
across(a:b, mean, na.rm = TRUE)
```

```
# Now
across(a:b, \(x) mean(x, na.rm = TRUE))
```

	TRT01PN	TRT01AN	TRTDUR	AVGDD	CUMDOSE	AGE	AGEGR1N	RACEN	BMIBL	HEIGHTBL
1	54	54	133.5	54	2322	77	2	1	24.2	162.85
	WEIGHTBL	EDUCLVL	DURDIS	VISNUMEN	MMSETOT					
1	66.7	12	36.25	11	19					

starts_with(), contains(), where(is.character) 등 다양한 조건으로 대상 열을 선택할 수 있어 매우 유연합니다.

11.5.2 rowwise()를 이용한 행 단위 처리

데이터의 각 행별로 계산이 필요할 때 사용합니다.

```
#
adsl_saf %>%
  rowwise(SUBJID) %>%
```

```
summarise(missing_cnt = sum(is.na(c_across(where(is.numeric))))) %>%
head()
```

`summarise()` has grouped output by 'SUBJID'. You can override using the
`.groups` argument.

```
# A tibble: 6 x 2
# Groups:   SUBJID [6]
  SUBJID missing_cnt
  <chr>         <int>
1 1015             0
2 1023             0
3 1028             0
4 1033             0
5 1034             0
6 1047             0
```

11.6 5. {purrr}와 map() 함수

{purrr} 패키지는 리스트 형태의 데이터를 다루는 강력한 도구입니다. map() 함수는 리스트의 각 요소에 함수를 적용합니다.

```
#
adsl_saf %>%
  split(.$TRT01A) %>%
  map(~ skimr::skim(.x))
```

\$Placebo

-- Data Summary -----

Name	Values
Number of rows	.x
Number of columns	86
	48

Column type frequency:	
character	28
Date	5
numeric	15

Group variables	None

-- Variable type: character -----

	skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
1	STUDYID	0	1	12	12	0	1	0
2	USUBJID	0	1	11	11	0	86	0
3	SUBJID	0	1	4	4	0	86	0
4	SITEID	0	1	3	3	0	16	0
5	SITEGR1	0	1	3	3	0	11	0
6	ARM	0	1	7	7	0	1	0
7	TRT01P	0	1	7	7	0	1	0
8	TRT01A	0	1	7	7	0	1	0
9	AGEGR1	0	1	3	5	0	3	0
10	AGEU	0	1	5	5	0	1	0
11	RACE	0	1	5	25	0	2	0
12	SEX	0	1	1	1	0	2	0
13	ETHNIC	0	1	18	22	0	2	0
14	SAFFL	0	1	1	1	0	1	0
15	ITTFL	0	1	1	1	0	1	0

16	EFFFL	0	1	1	1	0	2	0
17	COMP8FL	0	1	1	1	0	2	0
18	COMP16FL	0	1	1	1	0	2	0
19	COMP24FL	0	1	1	1	0	2	0
20	DISCONFL	0	1	0	1	58	2	0
21	DSRAEFL	0	1	0	1	78	2	0
22	DTHFL	0	1	0	1	84	2	0
23	BMIBLGR1	0	1	3	6	0	3	0
24	DURDSGR1	0	1	3	4	0	2	0
25	RFSTDTC	0	1	10	10	0	81	0
26	RFENDTC	0	1	10	10	0	80	0
27	DCDECOD	0	1	5	27	0	9	0
28	DCREASCD	0	1	5	18	0	10	0

-- Variable type: Date -----

	skim_variable	n_missing	complete_rate	min	max	median
1	TRTSDT	0	1	2012-07-09	2014-09-02	2013-07-12
2	TRTEDT	0	1	2012-09-01	2015-03-05	2013-11-22
3	DISONSDT	0	1	1998-06-13	2013-04-08	2010-08-17
4	VISIT1DT	0	1	2012-07-06	2014-08-29	2013-07-04
5	RFENDT	0	1	2012-09-02	2015-03-05	2013-11-22

n_unique

1	81
2	80
3	85
4	80
5	80

-- Variable type: numeric -----

	skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75
1	TRT01PN	0	1	0	0	0	0	0	0
2	TRT01AN	0	1	0	0	0	0	0	0
3	TRTDUR	0	1	149.	60.3	7	131	182	183
4	AVGDD	0	1	0	0	0	0	0	0
5	CUMDOSE	0	1	0	0	0	0	0	0
6	AGE	0	1	75.2	8.59	52	69.2	76	81.8
7	AGEGR1N	0	1	2.19	0.695	1	2	2	3
8	RACEN	0	1	1.09	0.292	1	1	1	1
9	BMIBL	0	1	23.6	3.67	15.1	21.2	23.4	25.6
10	HEIGHTBL	0	1	163.	11.5	137.	154	163.	171.
11	WEIGHTBL	0	1	62.8	12.8	34	53.6	60.6	74.2
12	EDUCLVL	0	1	12.6	2.95	6	12	12	14.8
13	DURDIS	0	1	42.6	30.2	7.2	24.3	35.3	50.1
14	VISNUMEN	0	1	10.7	2.38	4	11	12	12
15	MMSETOT	0	1	18.0	4.27	10	15	19.5	22

p100 hist

1	0
2	0
3	210
4	0
5	0
6	89
7	3
8	2
9	33.3
10	185.
11	86.2
12	21

```
13 183.
14 12
15 23
```

\$`Xanomeline High Dose`

-- Data Summary -----

	Values
Name	.x
Number of rows	84
Number of columns	48

Column type frequency:

character	28
Date	5
numeric	15

Group variables None

-- Variable type: character -----

	skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
1	STUDYID	0	1	12	12	0	1	0
2	USUBJID	0	1	11	11	0	84	0
3	SUBJID	0	1	4	4	0	84	0
4	SITEID	0	1	3	3	0	15	0
5	SITEGR1	0	1	3	3	0	11	0
6	ARM	0	1	20	20	0	1	0
7	TRT01P	0	1	20	20	0	1	0
8	TRT01A	0	1	20	20	0	1	0
9	AGEGR1	0	1	3	5	0	3	0
10	AGEU	0	1	5	5	0	1	0
11	RACE	0	1	5	32	0	3	0
12	SEX	0	1	1	1	0	2	0
13	ETHNIC	0	1	18	22	0	2	0
14	SAFFL	0	1	1	1	0	1	0
15	ITTFL	0	1	1	1	0	1	0
16	EFFFL	0	1	1	1	0	2	0
17	COMP8FL	0	1	1	1	0	2	0
18	COMP16FL	0	1	1	1	0	2	0
19	COMP24FL	0	1	1	1	0	2	0
20	DISCONFL	0	1	0	1	27	2	0
21	DSRAEFL	0	1	0	1	44	2	0
22	DTHFL	0	1	0	0	84	1	0
23	BMIBLGR1	0	1	3	6	0	3	0
24	DURDSGR1	0	1	3	4	0	2	0
25	RFSTDTC	0	1	10	10	0	78	0
26	RFENDTC	0	1	10	10	0	79	0
27	DCDECOD	0	1	9	27	0	7	0
28	DCREASCD	0	1	9	18	0	8	0

-- Variable type: Date -----

	skim_variable	n_missing	complete_rate	min	max	median
1	TRTSDT	0	1	2012-07-20	2014-07-01	2013-06-04
2	TRTEDT	0	1	2012-10-20	2014-12-30	2013-08-22
3	DISONSDT	0	1	2001-07-15	2013-02-04	2010-06-04
4	VISIT1DT	0	1	2012-07-10	2014-06-24	2013-05-28
5	RFENDT	0	1	2012-10-23	2014-12-30	2013-08-25
	n_unique					
1						78

```

2      78
3      82
4      75
5      79

```

```

-- Variable type: numeric -----
  skim_variable n_missing complete_rate   mean      sd    p0    p25    p50
1 TRT01PN           0           1    81        0    81    81    81
2 TRT01AN           0           1    81        0    81    81    81
3 TRTDUR            0           1   99.4     70.6     1    37.8   76.5
4 AVGDD             0           1   71.6     8.11    54    70.2   75.1
5 CUMDOSE           0           1 7551     5531.    54   2646   5778
6 AGE              0           1   74.4     7.89    56    70.8    76
7 AGEGR1N          0           1    2.08    0.585     1     2     2
8 RACEN            0           1    1.17    0.618     1     1     1
9 BMIBL            0           1   25.3     4.16   13.7   22.7   24.8
10 HEIGHTBL        0           1  166.     10.1  146.   158.   165.
11 WEIGHTBL        0           1   70.0    14.7   41.7   57.0   69.2
12 EDUCVL          0           1   12.5     2.92     6    11.8    12
13 DURDIS          0           1   40.5    24.7     2.2   23.9   36.0
14 VISNUMEN        0           1    8.98    2.87     4     7     9
15 MMSETOT         0           1   18.5     4.16    10    16    20
      p75    p100 hist
1    81    81
2    81    81
3   181.   200
4   76.9   78.6
5 13959  15417
6    80    88
7     2     3
8     1     6
9   27.8   34.5
10  173.   190.
11   80.3  108
12    15    20
13   52.4  135
14    12    12
15    22    24

```

\$`Xanomeline Low Dose`

```

-- Data Summary -----
                                     Values
Name                               .x
Number of rows                     84
Number of columns                   48

-----
Column type frequency:
  character                         28
  Date                             5
  numeric                          15

-----
Group variables                     None

```

```

-- Variable type: character -----
  skim_variable n_missing complete_rate min max empty n_unique whitespace
1 STUDYID       0           1  12  12    0     1           0
2 USUBJID       0           1  11  11    0    84           0
3 SUBJID        0           1   4   4    0    84           0

```


4 SITEID	0	1	3	3	0	17	0
5 SITEGR1	0	1	3	3	0	11	0
6 ARM	0	1	19	19	0	1	0
7 TRT01P	0	1	19	19	0	1	0
8 TRT01A	0	1	19	19	0	1	0
9 AGEGR1	0	1	3	5	0	3	0
10 AGEU	0	1	5	5	0	1	0
11 RACE	0	1	5	25	0	2	0
12 SEX	0	1	1	1	0	2	0
13 ETHNIC	0	1	18	22	0	2	0
14 SAFFL	0	1	1	1	0	1	0
15 ITTFL	0	1	1	1	0	1	0
16 EFFFLL	0	1	1	1	0	2	0
17 COMP8FL	0	1	1	1	0	2	0
18 COMP16FL	0	1	1	1	0	2	0
19 COMP24FL	0	1	1	1	0	2	0
20 DISCONFL	0	1	0	1	25	2	0
21 DSRAEFL	0	1	0	1	40	2	0
22 DTHFL	0	1	0	1	83	2	0
23 BMIBLGR1	0	1	3	6	0	3	0
24 DURDSGR1	0	1	3	4	0	2	0
25 RFSTDTC	0	1	10	10	0	77	0
26 RFENDTC	0	1	10	10	0	82	0
27 DCDECOD	0	1	5	27	0	7	0
28 DCREASCD	0	1	5	18	0	7	0

-- Variable type: Date -----

	skim_variable	n_missing	complete_rate	min	max	median
1	TRTSDT	0	1	2012-07-22	2014-05-22	2013-05-23
2	TRTEDT	0	1	2012-08-28	2014-11-20	2013-08-15
3	DISONSDT	0	1	2002-07-16	2013-09-16	2009-10-01
4	VISIT1DT	0	1	2012-07-08	2014-05-10	2013-05-13
5	RFENDT	0	1	2012-09-01	2014-11-20	2013-08-17

n_unique

1	77
2	80
3	83
4	77
5	82

-- Variable type: numeric -----

	skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50
1	TRT01PN	0	1	54	0	54	54	54
2	TRT01AN	0	1	54	0	54	54	54
3	TRTDUR	0	1	99.0	68.2	2	36.8	82.5
4	AVGDD	0	1	54	0	54	54	54
5	CUMDOSE	0	1	5347.	3680.	108	1984.	4455
6	AGE	0	1	75.7	8.29	51	71	77.5
7	AGEGR1N	0	1	2.25	0.618	1	2	2
8	RACEN	0	1	1.07	0.259	1	1	1
9	BMIBL	1	0.988	25.1	4.27	17.7	22.2	24.3
10	HEIGHTBL	0	1	163.	10.4	136.	158.	163.
11	WEIGHTBL	1	0.988	67.3	14.1	45.4	56.0	64.9
12	EDUCLVL	0	1	13.2	4.15	3	12	12
13	DURDIS	0	1	48.7	29.6	7.8	26.4	40.2
14	VISNUMEN	0	1	9.01	2.87	4	7	9.5
15	MMSETOT	0	1	17.9	4.22	10	14	18

p75 p100 hist

1	54	54
2	54	54
3	182.	212
4	54	54
5	9801	11448
6	82	88
7	3	3
8	1	2
9	27.8	40.1
10	170.	196.
11	77.4	106.
12	16	24
13	66.4	131.
14	12	12
15	22	24

11.7 6. 성능 최적화 (병렬 처리)

반복 작업이 매우 많고 시간이 오래 걸린다면, 여러 프로세서를 사용하는 병렬 처리를 고려할 수 있습니다. `{multidplyr}`, `{furrr}`, `{foreach}` 등의 패키지가 활용됩니다.

11.8 챌린지

미니 프로젝트 7에서 만든 `adsl_counts` 함수를 활용해 보세요. `adsl_saf` 데이터를 인종(RACE)별로 나누어(`split`) 각 그룹에 대해 치료군별 성별 빈도를 산출해 보세요.

Chapter 12

재현 가능한 예제와 {reprex}

재현 가능한 예제(Reproducible Example, 줄여서 **Reprex**)를 만드는 법을 배웁니다. 코드가 제대로 작동하지 않을 때 다른 사람에게 도움을 요청하는 가장 효율적인 방법은 상대방이 내 코드를 그대로 복사해서 바로 실행해볼 수 있도록 준비하는 것입니다.

12.1 목표

- {reprex} 패키지 이해 및 설치
- 최소 재현 가능 예제(Minimal Reproducible Example) 생성 원칙
- 스크린샷 대신 실행 가능한 코드 공유하기

```
# install.packages("reprex")
library(reprex)
library(tidyverse)
```

Warning: package 'ggplot2' was built under R version 4.4.3

Warning: package 'tibble' was built under R version 4.4.3

Warning: package 'purrr' was built under R version 4.4.2

Warning: package 'lubridate' was built under R version 4.4.2

-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --

```
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    3.5.2      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.0.4
```

-- Conflicts ----- tidyverse_conflicts() --

x dplyr::filter() masks stats::filter()

x dplyr::lag() masks stats::lag()

i Use the conflicted package (<<http://conflicted.r-lib.org/>>) to force all conflicts to become errors

```
library(rio)
```

12.2 1. Reprex란 무엇인가?

reprex는 작은 코드 조각을 실행 결과와 함께 보기 좋게 패키징해주는 도구입니다. 도움을 주는 사람이 코드를 다시 포맷하거나, 누락된 패키지를 추측하거나, 데이터를 찾기 위해 고생할 필요가 없도록 해줍니다.

중요: R 세션의 에러 메시지를 스크린샷으로 찍어 보내지 마세요! 스크린샷 속의 코드는 실행해볼 수 없으므로 도움을 주는 사람이 일일이 코드를 다시 타이핑해야 합니다. 이는 매우 비효율적입니다.

12.3 2. 기본 사용법

가장 간단한 예시입니다. 아래 코드를 복사(Ctrl + C)한 다음, 콘솔 창에서 `reprex()`를 입력하고 실행해 보세요.

```
(y <- 1:4)
```

```
[1] 1 2 3 4
```

```
mean(y)
```

```
[1] 2.5
```

잠시 후 RStudio의 **Viewer** 창에 실행 결과가 포함된 코드 블록이 생성됩니다. 이 내용은 자동으로 클립보드에도 복사되므로, 그대로 이메일이나 Teams 메시지 창에 붙여넣기(Ctrl + V)만 하면 됩니다.

12.4 3. 재현 가능한 예제에 포함되어야 할 내용

1. **배경 정보:** 무엇을 하려고 했는지, 현재 상황은 어떤지에 대한 짧은 설명
2. **완전한 설정:** 필요한 모든 `library()` 호출과 데이터 로드 코드 포함
3. **최소한의 코드:** 에러를 재현하는 데 꼭 필요한 코드만 남기고 나머지는 제거
4. **데이터 포함:** 상대방이 접근할 수 없는 로컬 파일 대신, `tribble()` 등을 이용해 만든 작은 샘플 데이터 사용

12.4.1 팁: `tribble()`을 이용한 샘플 데이터 생성

도움을 주는 사람이 내 PC의 데이터셋에 접근할 수 없으므로, 코드 내에서 즉석으로 데이터를 만드는 것이 좋습니다.

```
# SAS datalines
myData <- tribble(
  ~Treatment, ~value,
  "Placebo", 1,
  "Placebo", 2,
  "Active", 3,
  "Active", 4
)

myData
```

```
# A tibble: 4 x 2
  Treatment value
  <chr>      <dbl>
1 Placebo      1
2 Placebo      2
3 Active       3
4 Active       4
```

이처럼 작고 간결한 데이터를 사용하면 문제를 훨씬 더 명확하게 전달할 수 있습니다.

12.5 4. 주의사항

- `rm(list = ls())`로 코드를 시작하지 마세요. 도움을 주는 사람의 현재 작업 환경을 모두 지워버릴 수 있습니다.
- `setwd("C:/Users/ /Documents")`와 같은 로컬 경로를 포함하지 마세요. 다른 사람의 컴퓨터에서는 작동하지 않습니다.
- 문제와 관련 없는 패키지나 코드는 과감히 삭제하세요.

12.6 챌린지

아래 코드는 에러를 발생시키거나 원하는 결과를 보여주지 않습니다. 다른 사람이 이 문제를 바로 파악하고 해결할 수 있도록, 위에서 배운 원칙에 따라 **최소 재현 가능 예제**를 만들어 보세요.

```
library(tidyverse)
library(rio)

adlb <- import("./data/adlbh.xpt")

BICARB <- adlb %>%
  filter(SAFFL == "Y") %>%
  filter(PARAMCD == "BASO")

ggplot(data = BICARB,
       mapping = aes(x = ADY, y = AVAL)) +
  geom_point(aes(colour = "THERAPY1"))
```

Chapter 13

디버깅 (Debugging)

코드의 에러를 찾고 수정하는 다양한 디버깅 전략을 배웁니다.

13.1 디버깅 전략

1. **꼐다가 다시 켜기**: 환경(Environment)을 깨끗이 비우고 다시 시작하기
2. **환경 이해하기**: 전역(Global) 환경과 지역(Local) 환경 구분하기
3. **도구 활용**: `debugonce()`, `browser()`, RStudio 중단점(Breakpoint) 사용하기
4. **추적**: `traceback()`, `options(error = recover)` 활용하기

참고 비디오: [R 디버깅 기초](#)

13.2 1. 깨끗한 환경 유지하기

R 세션을 오래 사용하다 보면 수많은 객체가 쌓이게 됩니다. 이전에 만든 객체가 현재 작업에 의도치 않은 영향을 줄 수 있으므로, 주기적으로 환경을 정리하고 R을 재시작하는 것이 좋습니다.

- **빗자루 아이콘**: Environment 탭의 빗자루 아이콘을 클릭하면 현재 세션의 모든 객체가 삭제됩니다.
- **재현성 확보**: 스크립트를 처음부터 끝까지 실행했을 때 오류가 없어야 합니다. RStudio 설정에서 세션 종료 시 작업 공간을 저장하지 않도록 설정(Save workspace to .RData on exit: Never)하는 것을 권장합니다.

13.3 2. R 다시 시작하기 (Restart R)

환경을 비우는 것만으로 해결되지 않는 설정(로딩된 패키지, 옵션 등)이 있습니다. 이럴 때는 R 자체를 다시 시작해야 합니다.

- **방법**: RStudio 메뉴의 `Session -> Restart R` 또는 단축키 `Ctrl+Shift+F10`.
- 특히 새로운 패키지를 설치하기 전이나 후에 세션을 초기화하는 습관을 들이는 것이 좋습니다.

13.4 3. 실행 환경의 이해

함수를 실행하면 R은 **실행 환경 (Execution Environment)**이라는 별도의 공간을 만듭니다. 함수 내부에서 생성된 객체는 함수 종료와 함께 사라지며, 전역 환경(Global Environment)에 영향을 주지 않습니다.

```
myFunction <- function(x) {  
  a <- 3  
  x + a  
}
```

```
myFunction(2) # : 5
```

```
[1] 5
```

```
# 'a' 'a' .
```

- 함수 내부에서 전역 변수를 직접 참조하는 것은 좋지 않은 습관입니다. 필요한 값은 인수를 통해 전달하세요.

13.5 4. debugonce() 활용하기

함수 내부에서 어떤 일이 일어나는지 한 줄씩 확인하고 싶을 때 사용합니다.

```
debugonce(myFunction)
myFunction(2)
```

이 코드를 실행하면 RStudio는 디버그 모드로 진입합니다. - **Next (n)**: 다음 줄 실행 - **Continue (c)**: 남은 코드 모두 실행 - **Stop (Q)**: 디버그 모드 종료

Environment 탭이 현재 함수의 실행 환경으로 바뀌어 내부 변수값들을 실시간으로 확인할 수 있습니다.

13.6 5. browser()와 중단점(Breakpoint)

- **browser()**: 코드 중간에 browser() 함수를 삽입하면 실행 중에 해당 지점에서 멈추고 디버그 모드로 들어갑니다.
- **중단점**: 스크립트 에디터의 행 번호 왼쪽을 클릭하여 빨간 점(중단점)을 찍을 수 있습니다. 코드를 Source 실행할 때 해당 지점에서 멈춥니다.

13.7 6. 에러 추적: traceback()과 recover()

에러가 발생했을 때 어디서부터 잘못되었는지 역추적합니다.

- **traceback()**: 에러 직후 호출하면 어떤 함수 호출 과정을 거쳐 에러가 났는지 보여줍니다.
- **options(error = recover)**: 에러 발생 시 즉시 멈추고, 어느 단계의 환경을 조사할지 선택할 수 있게 해줍니다. (조사 후에는 options(error = NULL)로 되돌리세요.)

13.8 챌린지

아래 adsl_counts 함수에는 에러가 포함되어 있습니다. 위에서 배운 디버깅 도구들을 사용하여 어디서 에러가 발생하는지 찾아보세요.

```
adsl_counts <- function(dataFile) {
  adsl_saf <- dataFile %>% filter(SAFFL == "Y")

  Big_N_cnt <- adsl_saf %>%
    group_by(TRT01AN, TRT01A) %>%
    summarise(N = n(), .groups = 'drop')

  small_n_cnt <- adsl_saf %>%
    group_by(TRT01AN, TRT01A, SEX) %>%
    summarise(name = "n", .groups = 'drop') # ?

  adsl_mrg_cnt <- small_n_cnt %>%
    left_join(Big_N_cnt, by = c("TRT01AN", "TRT01A")) %>%
    mutate(perc = round((n/N)*100, 1)) # 'n' ?
```

```
    return(adsl_mrg_cnt)
}

# : debugonce(adsl_counts) .
```


Chapter 14

Tidy Evaluation (타이디 평가)

Tidy evaluation (줄여서 **tidyeval**)은 dplyr이나 ggplot2 같은 패키지에서 따옴표 없이 변수명을 사용할 수 있게 해주는 마법 같은 도구입니다. 이 기술을 활용하면 더 유연하고 강력한 사용자 정의 함수를 만들 수 있습니다.

14.1 1. tidyeval이란 무엇인가?

우리는 이미 `mtcars %>% select(mpg, cyl)` 처럼 변수명을 따옴표 없이 사용해 왔습니다. Tidyeval은 R에서 **메타프로그래밍(Metaprogramming)**을 가능하게 하는 기술입니다. SAS 매크로와 유사한 역할을 한다고 이해할 수 있습니다.

14.2 2. 평가된 인수와 인용된 인수

- **평가된 인수(Evaluated)**: 함수에 전달되기 전 미리 계산된 값 (예: `log(10)`)
- **인용된 인수(Quoted)**: 변수명 그 자체를 전달하고, 나중에 데이터의 맥락에서 해석함

```
mpg <- 5020
#      mpg(5020)      mpg
mtcars %>% select(mpg) %>% head(2)
```

```
      mpg
Mazda RX4      21
Mazda RX4 Wag  21
```

14.3 3. 열 충돌(Column Collision) 방지

데이터셋 내부의 변수명과 외부 환경의 변수명이 같을 때 발생하는 문제를 방지하기 위해 `.data`와 `.env` 대명사를 사용할 수 있습니다.

```
df <- data.frame(x = 1, y = 2)
x <- 100

df %>%
  mutate(z1 = y / x) %>%      # x
  mutate(z2 = .data$y / .env$x) #
```

```
  x y z1  z2
1 1 2  2 0.02
```

14.4 4. 동적 점 (Dynamic Dots, ...)

함수에서 ...을 사용하면 개수에 상관없이 여러 인수를 한 번에 받을 수 있습니다.

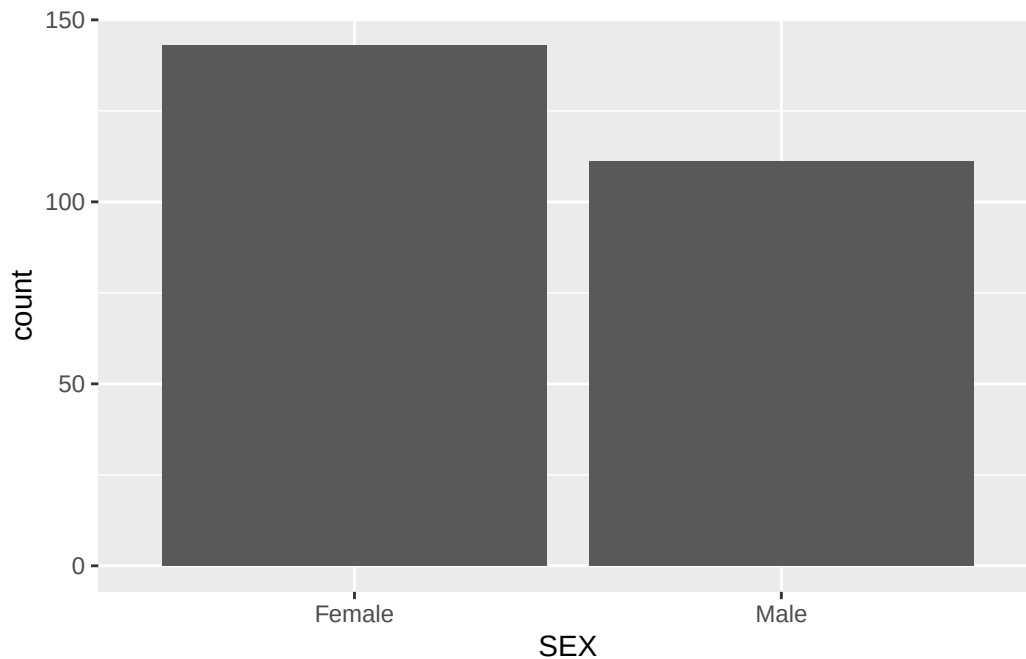
```
mySummary <- function(myData, ...) {  
  myData %>%  
    group_by(TRT01AN, TRT01A) %>%  
    summarise(mean = round(mean(AGE), ...), .groups = 'drop')  
}  
  
#      round()      (digits = 1)  
mySummary(adsl_saf, digits = 1)
```

```
# A tibble: 3 x 3  
  TRT01AN TRT01A      mean  
  <dbl> <chr>      <dbl>  
1      0 Placebo      75.2  
2     54 Xanomeline Low Dose 75.7  
3     81 Xanomeline High Dose 74.4
```

14.5 5. Embrace 연산자 ({{ }})

데이터셋의 변수명을 함수의 인수로 받고 싶을 때 가장 쉬운 방법은 { }를 사용하는 것입니다. 이를 **Embrace** 한다고 표현합니다.

```
plot_bar <- function(data, var) {  
  data %>%  
    ggplot(aes({{ var }})) +  
    geom_bar()  
}  
  
plot_bar(adsl_saf, SEX)
```



14.6 6. Walrus 연산자 (:=)

함수 내부에서 새로운 변수 이름을 동적으로 지정하고 싶을 때는 = 대신 := (Walrus 연산자)를 사용해야 합니다.

```
data(mpg)
summary_mean <- function(df, summary_var, new_name) {
  df %>%
    summarize({{ new_name }} := mean({{ summary_var }}))
}

summary_mean(mpg, cty, mean_cty)
```

```
# A tibble: 1 x 1
  mean_cty
  <dbl>
1      16.9
```

14.7 7. 기호를 문자열로 변환 (as_name)

드물게 인수로 받은 변수명을 실제 문자열("variable_name")로 바꿔서 사용해야 할 때가 있습니다. 이때 rlang::as_name(enquo(var)) 형식을 사용합니다.

```
get_var_name <- function(data, var) {
  var_str <- rlang::as_name(enquo(var))
  message("      : ", var_str)
}

get_var_name(adsl_saf, RACE)
```

```
      : RACE
```

14.8 챌린지

미니 프로젝트 7에서 개발한 plot_ALT 함수에 tidyeval 기술을 적용해 보세요. { }와 ...을 사용하여 더 유연하게 작동하도록 개선해 보세요.

```
library(ggdist)

plot_ALT <- function(data, x_var, y_var, ...) {
  data %>%
    ggplot(aes(x = {{ x_var }}, y = as.factor({{ y_var }}))) +
    ggdist::stat_dotsinterval() +
    labs(...) # labs      ...
}

#
# import("../data/adlbc.xpt") %>% filter(PARAMCD == "ALT") %>%
#   plot_ALT(x_var = LBSTRESN, y_var = VISITNUM, x = "ALT ", y = " ")
```

Chapter 15

단위 테스트 (Unit Testing)

코드를 테스트하는 것은 번거로울 수 있지만, 코드의 품질을 높이고 예기치 않은 버그를 방지하는 가장 확실한 방법입니다. R의 대표적인 테스트 패키지인 `{testthat}`을 사용하는 법을 배웁니다.

15.1 목표

- 자동화된 테스트의 이점 이해
- `{testthat}` 패키지의 기본 사용법 익히기
- 기대치(Expectation) 함수 활용하기
- 테스트 가능한 코드 작성하기

15.2 1. 테스트의 이점

우리는 보통 함수를 만든 뒤 콘솔에서 몇 가지 값을 넣어보며 눈으로 결과를 확인합니다. 하지만 시간이 흐른 뒤 코드를 수정했을 때, 예전에 잘 작동하던 기능이 여전히 잘 돌아가는지 매번 수동으로 확인하는 것은 불가능에 가깝습니다.

자동화된 단위 테스트(Automated Unit Testing)를 사용하면: - 코드 변경 시 기존 기능이 망가졌는지 즉시 확인 가능합니다. - 에러가 발생한 위치를 더 빨리 찾을 수 있습니다. - 테스트 코드 자체가 훌륭한 문서 역할을 합니다. - 테스트하기 쉬운 코드를 짜게 되므로 전체적인 설계가 개선됩니다.

15.3 2. `{testthat}`의 기대치(Expectation)

기대치는 테스트의 최소 단위로, “함수의 결과가 내가 생각한 값/타입과 일치하는가?”를 확인합니다. 모든 함수는 `expect_`로 시작합니다.

- `expect_equal()`: 값이 같은지 확인
- `expect_type()`: 데이터 타입이 맞는지 확인
- `expect_length()`: 벡터의 길이가 맞는지 확인
- `expect_error()`: 에러가 발생하는지 확인

15.4 3. 테스트 가능한 코드 작성 예시

테스트하기 좋은 함수는 하나의 명확한 작업만 수행해야 합니다.

```
#
wrangle_data <- function(data) {
  res <- data %>%
    filter(SAFFL == "Y") %>%
    select(STUDYID, USUBJID, SEX, TRT01A, SAFFL)
```

```

    return(res)
  }

#
calculate_counts <- function(data) {
  res <- data %>%
    group_by(TRT01A, SEX) %>%
    count(name = "n")
  return(res)
}

```

15.5 4. 테스트 코드 작성하기

실제 데이터를 사용하여 위 함수들이 잘 작동하는지 테스트해 봅니다.

```

#
test_df <- tribble(
  ~STUDYID, ~USUBJID, ~SEX, ~TRT01A, ~SAFFL,
  "S01", "001", "M", "Placebo", "Y",
  "S01", "002", "F", "Placebo", "Y",
  "S01", "003", "M", "Active", "N"
)

# 1. wrangle_data
test_that("wrangle_data", {
  res <- wrangle_data(test_df)

  # SAFFL == 'Y' 2
  expect_equal(nrow(res), 2)
  # 5
  expect_length(names(res), 5)
  # 'N'
  expect_false("N" %in% res$SAFFL)
})

```

Test passed

```

# 2. calculate_counts
test_that(" ", {
  res <- calculate_counts(wrangle_data(test_df))

  # Placebo (M) 1
  expect_equal(res %>% filter(TRT01A == "Placebo", SEX == "M") %>% pull(n), 1)
})

```

Test passed

15.6 5. 테스트 조직화 (Folder Structure)

규모가 큰 프로젝트에서는 테스트 코드를 별도의 파일로 관리합니다. 통상적인 구조는 다음과 같습니다.

- R/: 실제 함수가 담긴 파일들
- tests/testthat/: test-로 시작하는 테스트 파일들
- tests/testthat.R: 모든 테스트를 한 번에 실행하는 스크립트

15.7 챌린지

미니 프로젝트 7에서 작성한 `ads1_counts` 함수에 대한 테스트 코드를 작성해 보세요. 백분율(`perc`) 계산이 정확한지, 결측치가 포함된 데이터가 들어왔을 때 어떻게 반응하는지 등을 확인하는 테스트 케이스를 3개 이상 만들어 보세요.

Chapter 16

코드 스타일 (Code Style)

코드 스타일은 동료가 내 코드를 읽고 이해하는 데 도움을 줄 뿐만 아니라(QC 과정에 필수적), 미래의 나 자신이 과거에 무엇을 했는지 기억하는 데에도 큰 도움이 됩니다. 프로그래밍 표준과 좋은 관행을 익혀 제3자도 이해할 수 있는 코드를 작성하는 법을 배웁니다.

16.1 핵심 요약 (TL;DR)

- **일관된 스타일 유지:** `{tidyverse}` 스타일 가이드를 권장합니다. RStudio에는 이를 쉽게 구현할 수 있는 도구들이 있습니다.
- **환경 정보 기록:** `session_info()` 등을 사용하여 작업 당시의 R 버전 및 패키지 정보를 반드시 남기세요.
- **문서화:** 별도의 패키지를 만드는 것이 아니라면, `{rmarkdown}`이나 Quarto를 사용하여 코드와 설명을 함께 작성하고 HTML로 렌더링하세요.
- **중복 배제 (DRY):** 동일한 작업을 반복하지 마세요. 함수를 작성하고 문서화하여 테스트하세요.

16.2 1. 들여쓰기 정리 (단축키: Ctrl + I)

코드의 가독성을 높이는 가장 쉬운 방법은 들여쓰기를 맞추는 것입니다. RStudio에서 정리가 필요한 코드를 선택한 뒤 Ctrl + I를 눌러보세요.

16.3 2. 코드 다시 포맷하기 (단축키: Ctrl + Shift + A)

단순 들여쓰기를 넘어, 줄 바꿈이나 연산자 앞뒤 공백 등을 tidyverse 스타일에 맞춰 자동으로 정리해 줍니다.

16.4 3. 일관성 있는 스타일의 중요성

어떤 스타일 가이드를 선택하든 **일관성**이 가장 중요합니다.

1. **할당 연산자:** = 보다는 <- 사용을 권장합니다.
2. **명명 규칙:** snake_case (예: big_n_cnt) 또는 camelCase 중 하나를 골라 일관되게 사용하세요.
3. **공백 사용:** <- 주위와 쉼표 뒤에는 항상 공백을 넣으세요.
4. **줄 바꿈:**
 - 파이프 연산자(%>% 또는 |>) 뒤에는 줄 바꿈을 하세요.
 - ggplot2의 레이어(+) 뒤에도 줄 바꿈을 하세요.
 - 인수가 많은 함수는 여러 줄로 나누어 작성하세요.

16.5 4. 주석 (Comments) 점검

주석은 코드가 무엇(WHAT)을 하는지보다, 왜(WHY) 그렇게 작성했는지에 대한 맥락을 설명해야 합니다. 코드를 수정했다면 주석도 그에 맞게 업데이트되었는지 확인하세요.

16.6 5. 불필요한 라이브러리 로드 금지

사용하지 않는 패키지를 로드하면 충돌이 발생하거나 메모리를 낭비할 수 있습니다. 분석이 끝난 뒤에는 실제로 사용한 패키지만 남겨두고 정리하세요.

16.7 6. 중복 배제 (DRY: Don't Repeat Yourself)

비슷한 작업을 수행하는 코드 문치가 세 번 이상 반복된다면, 함수로 만들 시점입니다. 복사-붙여넣기는 오타나 에러의 원인이 되기 쉽습니다.

```
#      :
BIGNcnt_sex <- adsl_saf %>% group_by(TRT01A) %>% count()
BIGNcnt_age <- adsl_saf %>% group_by(TRT01A) %>% count() #

#      :
big_n <- adsl_saf %>% group_by(TRT01A) %>% count(name = "N")
```


Chapter 17

R 코드 스니펫 (Code Snippets)

코드 스니펫(Code Snippets)은 자주 사용하는 코드 문치를 빠르게 삽입할 수 있게 해주는 텍스트 매크로입니다. (SAS Studio의 스니펫 옵션과 유사합니다). 반복되는 코드를 일일이 타이핑하지 않아도 되므로 시간을 대폭 절약할 수 있습니다.

17.1 1. 개요

RStudio 메뉴의 Tools -> Global Options -> Code -> Edit Snippets에서 등록된 스니펫을 확인하고 편집할 수 있습니다.

17.2 2. 스니펫 사용법

스크립트나 RMarkdown 파일에서 스니펫의 이름을 입력하기 시작하면 자동 완성 목록이 나타납니다. 예를 들어, fun을 입력하고 TAB 키를 누르면 아래와 같은 함수 구조가 즉시 삽입됩니다.

```
# 'fun'      TAB
name <- function(variables) {

}
```

- **자리 표시자(Placeholder):** 스니펫 삽입 후 변수명 등이 하이라이트됩니다. 내용을 입력하고 TAB 키를 누르면 다음 입력 위치로 바로 이동합니다.
- **수동 실행:** 자동 완성 목록이 뜨지 않는다면 Shift + TAB을 눌러 강제로 스니펫을 호출할 수 있습니다. (RMarkdown 모드에서는 이 방식이 필요할 수 있습니다.)

17.3 3. 자주 쓰이는 내장 스니펫

1. lib: library() 호출
2. req: require() 호출
3. src: source() 호출
4. df: 데이터 프레임 정의
5. if, el, ei: 조건문 (if, else, else if)
6. apply, lapply, sapply: apply 계열 함수

17.4 4. 나만의 스니펫 만들기

자주 사용하는 패키지 로드 세트나 전용 그래프 템플릿을 스니펫으로 만들어두면 매우 편리합니다.

- **편집 방법:** usethis 패키지의 edit_rstudio_snippets(type = "r") 함수를 실행하여 편집창을 열 수 있습니다.

- 문법:
 - snippet 키워드로 시작하고 이름을 지정합니다.
 - 다음 줄부터는 반드시 **TAB**으로 들여쓰기를 해야 합니다. (공백 사용 금지)
 - `${1:name}` 형식으로 자리 표시자를 지정할 수 있습니다.

17.4.1 예시: 스크립트 헤더 템플릿

프로그램의 목적, 작성자, 날짜 등을 기록하는 표준 헤더를 만들어 보세요.

```
snippet myheader
#####
#   : ${1: _ }
#   : ${2: }
#   : 2026-02-20
#   : ${3:data.xpt}
#####
```

이제 스크립트 상단에서 myheader를 입력하고 TAB을 누르면 미리 정의된 양식이 나타납니다.

17.5 5. 그래프 템플릿 스니펫

자주 사용하는 ggplot2 옵션들을 스니펫으로 만들어두면 시각화 작업이 훨씬 빨라집니다.

```
#   :
ggplot(${1:data}, aes(x = ${2:x_var}, y = ${3:y_var})) +
  geom_col(fill = "steelblue") +
  theme_minimal() +
  labs(title = "${4: }", x = "${5:x}", y = "${6:y_var}")
```

17.6 참고 자료

- [RStudio 공식 스니펫 가이드](#)
- [Posit 고객지원: Code Snippets](#)